

Estructuras de Datos

Clase 17 – Grafos: Definiciones, ADT, Representaciones



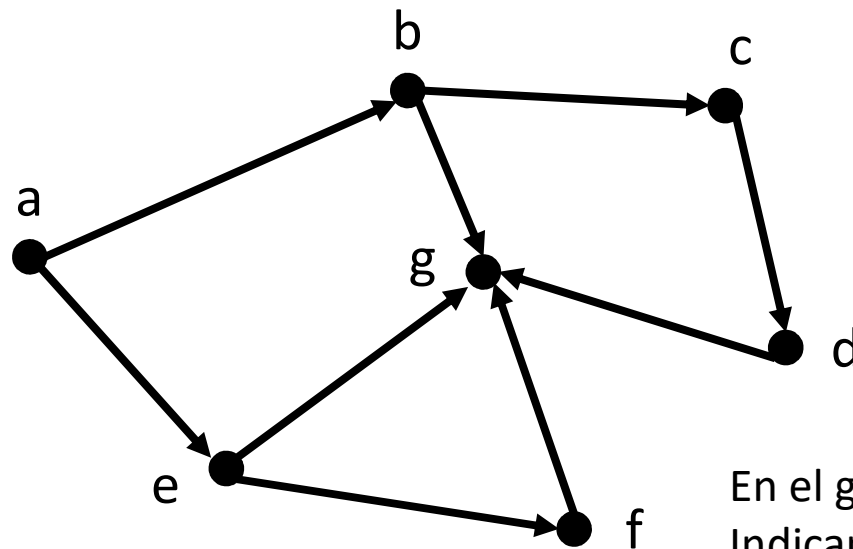
Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Definición: Grafo dirigido o digrafo

- Un grafo $G=(V,E)$ es un conjunto V de vértices (o nodos) y un conjunto $E \subseteq V \times V$ de aristas dirigidas (o arcos dirigidos).
- Intuitivamente E permite representar una relación entre elementos de V .



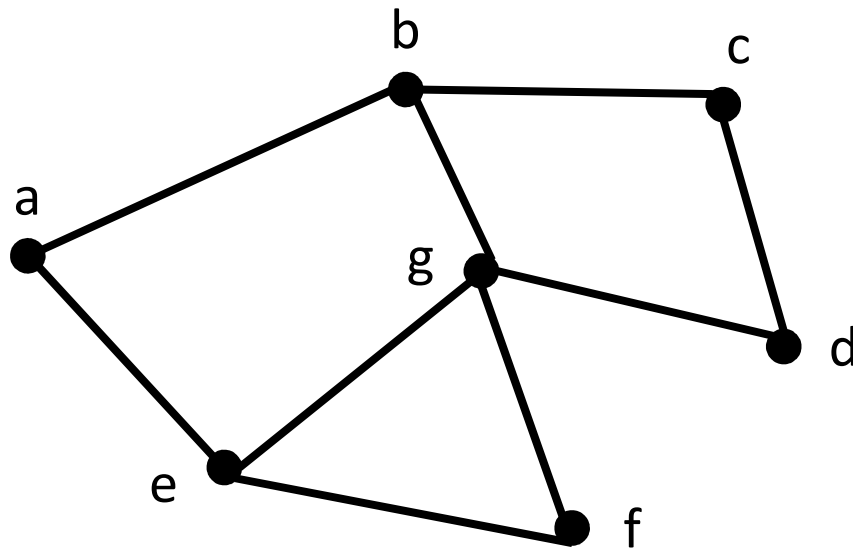
$V = \{ a, b, c, d, e, f, g \}$

$E = \{ (a,b), (b,c), (c,d), (d,g), (e,g), (e,f), (f,g), (a,e), (b,g) \}$

En el grafo dirigido, el arco (u,v) es un par ordenado Indicando una flecha del vértice u al vértice v .

Definición: Grafo no dirigido

- Un grafo $G=(V,E)$ es un conjunto V de vértices (o nodos) y un conjunto $E \subseteq V \times V$ de aristas (o arcos).
- Intuitivamente E permite representar una relación (simétrica) entre elementos de V .



$V = \{ a, b, c, d, e, f, g \}$

$E = \{ (a,b), (b,c), (c,d), (d,g), (e,g), (e,f), (f,g), (a,e), (b,g) \}$

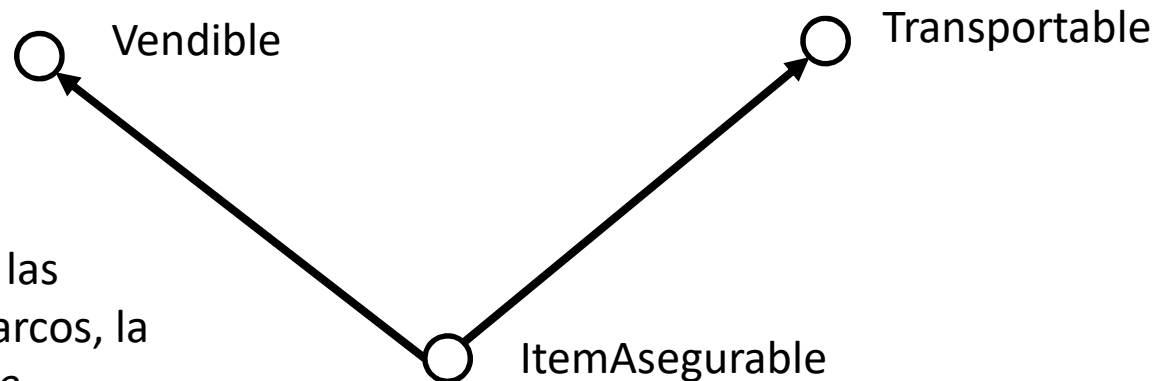
En el grafo no dirigido, el arco (u,v) en realidad es un conjunto $\{u,v\}$ ya que (u,v) es lo mismo que (v,u) .

Ejemplo de grafo dirigido: Grafo de herencia de interfaces en Java.

```
// Vendible.java  
public interface Vendible {  
    public String descripcion();  
    public int precioLista();  
}
```

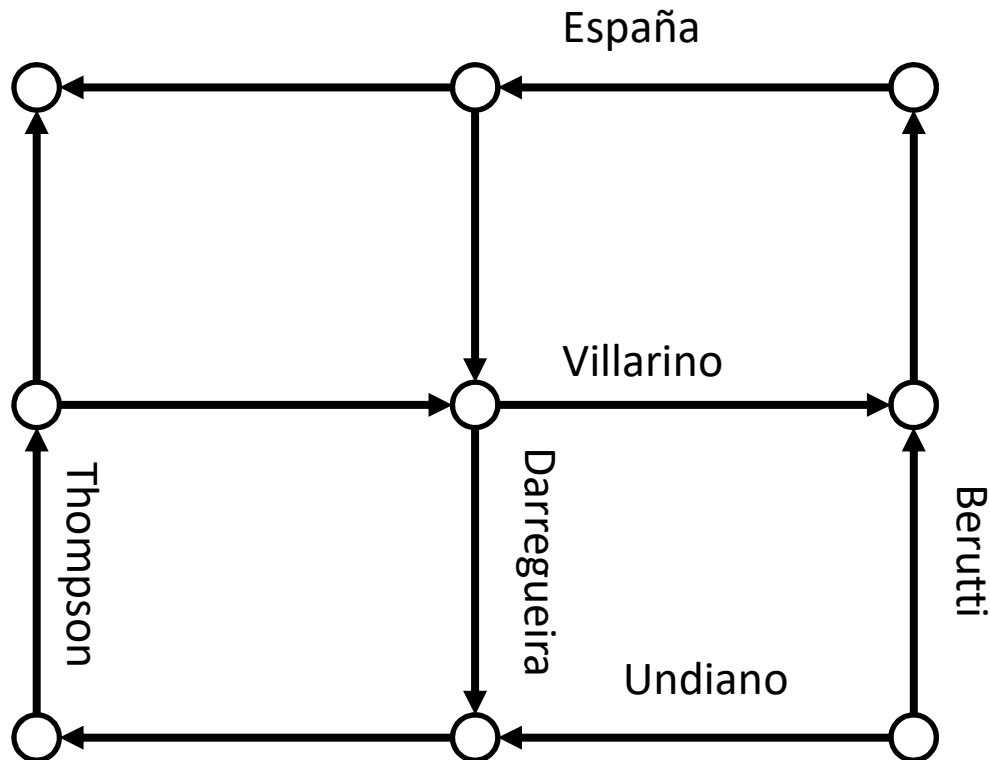
```
// Transportable.java  
public interface Transportable {  
    public int peso();  
    public boolean peligroso();  
}
```

```
public interface ItemAsegurable extends Vendible, Transportable {  
    public int valorAsegurado();  
}
```

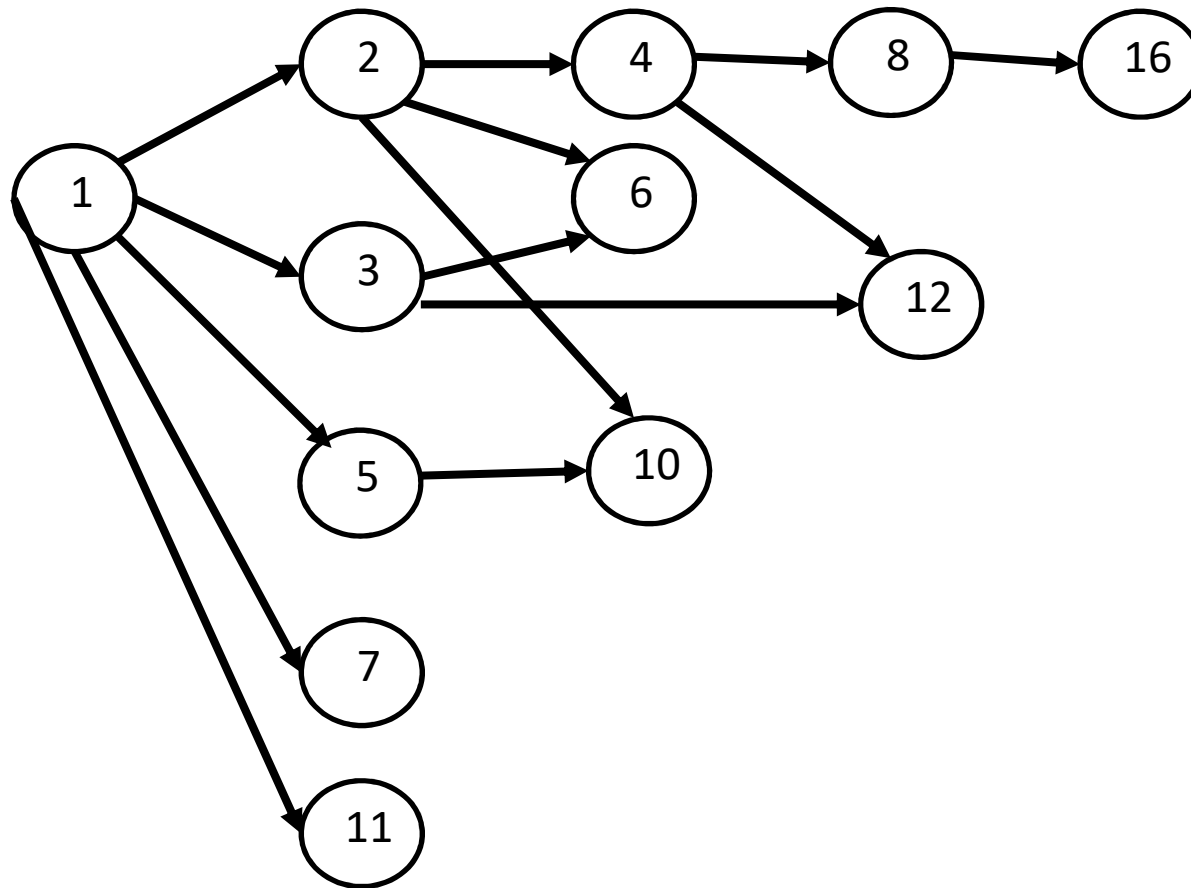


Los vértices son las interfaces y los arcos, la relación *extiende*.

Ejemplo de grafo dirigido: Grafo donde los vértices son esquinas de una ciudad y los arcos son cuadas de una mano.



Ejemplo de grafo dirigido: Grafo donde los vértices son números enteros y los arcos indican qué número divide a otro. E.g. como “2 divide a 6”, habrá un arco (2,6).



Ejemplo de grafo no dirigido: Grafo de colaboración entre coautores científicos.

**An Argumentative Approach to Assessing Safety in Medical Device Software
using Defeasible Logic Programming**

S.A. Gómez¹, A. Groza² and C.I. Chesñevar¹

**Defeasible Argumentation Support for an Extended BDI
Architecture***

Nicolás D. Rotstein, Alejandro J. García, and Guillermo R. Simari

Logical Models of Argument

CARLOS IVÁN CHESÑEVAR and ANA GABRIELA MAGUITMAN

Universidad Nacional del Sur

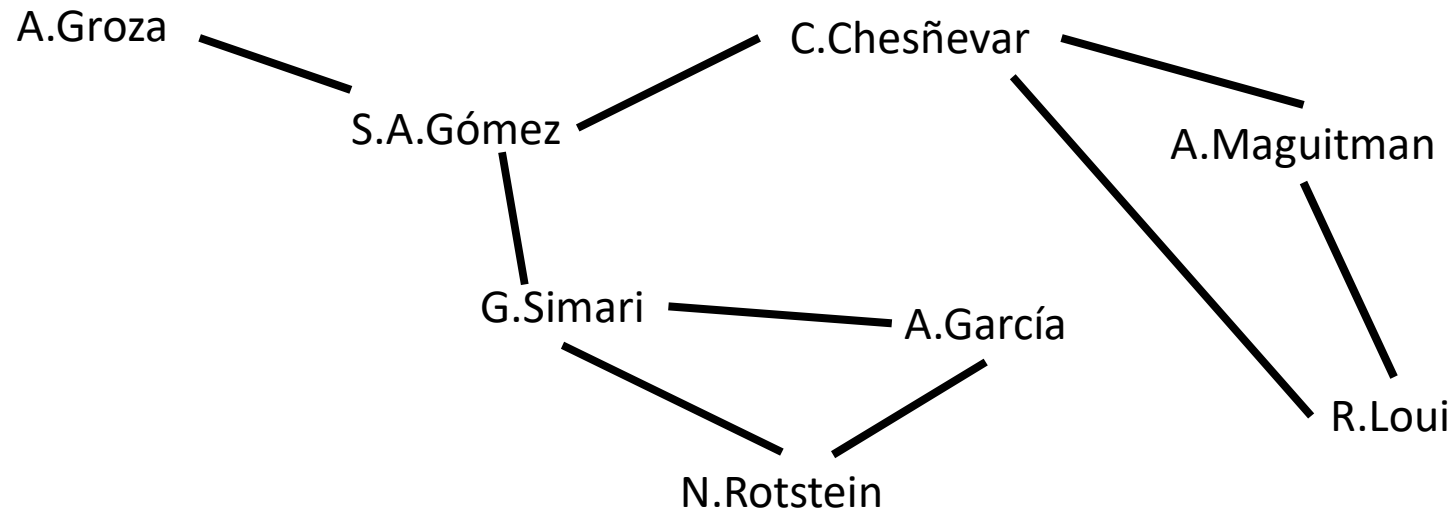
AND

RONALD PRESCOTT LOUI

Washington University

Sergio Alejandro Gómez, Guillermo Ricardo Simari:

Merging of Ontologies Using Belief Revision and Defeasible Logic Programming. 16-28



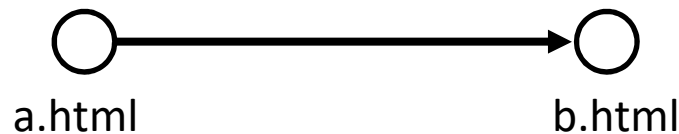
Ejemplo de grafo dirigido: World Wide Web = Colección documentos HTML enlazados con hipervínculos inventada por Tim Berners-Lee en 1989.



```
<html>
<head><title>a</title></head>
<body>
<a href="b.html">b</a>
</body>
</html>
```

hipervínculo

El documento a.html contiene un hipervínculo al documento b:



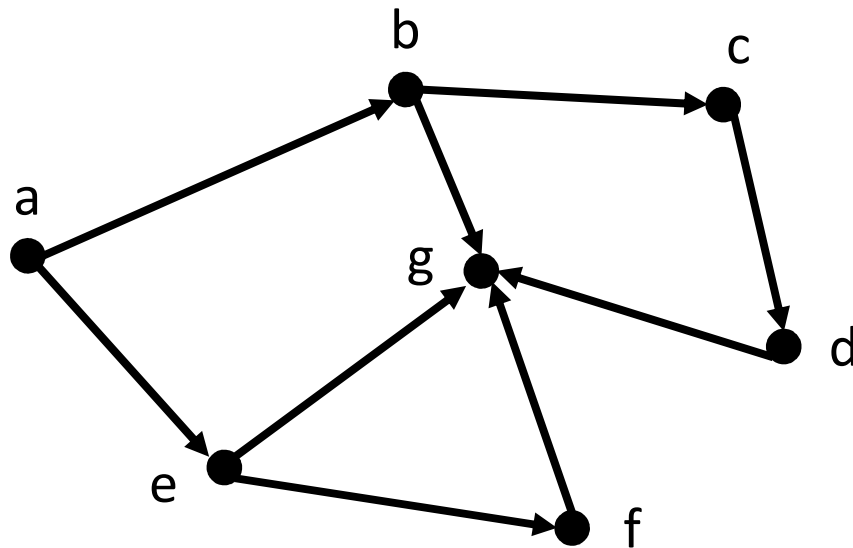
Cada documento es un vértice identificado con una URL y los hipervínculos dentro de un documento son los arcos dirigidos.

Arcos emergentes y grado de emergencia

$\text{ArcosEmergentes}(v) = \{ (v,y) : (v,y) \in E \}$

$\text{GradoDeEmergencia}(v) = \text{outdegree}(v) =$
 $\# \text{ArcosEmergentes}(v)$

Hojas del dígrafo = vértices con grado de emergencia 0.



$\text{ArcosEmergentes}(b) =$
 $\{ (b,c), (b,g) \}$

$\text{Outdegree}(b) = 2$

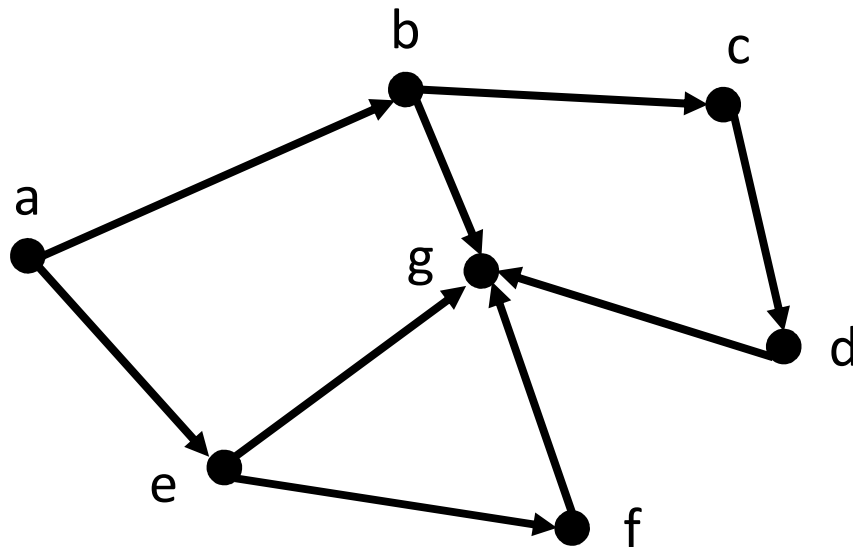
El vértice g es una hoja del dígrafo

Arcos incidentes y grado de incidencia

$\text{ArcosIncidentes}(v) = \{ (x,v) : (x,v) \in E \}$

$\text{GradoDeIncidencia}(v) = \text{indegree}(v) =$
 $\# \text{ArcosIncidentes}(v)$

Raíces del dígrafo = vértices con grado de incidencia 0.



$\text{ArcosIncidentes}(g) =$
 $\{ (b,g), (e,g), (d,g), (f,g) \}$
 $\text{Indegree}(g) = 4$

$\text{ArcosIncidentes}(a) = \{ \}$
 $\text{Indegree}(a) = 0$

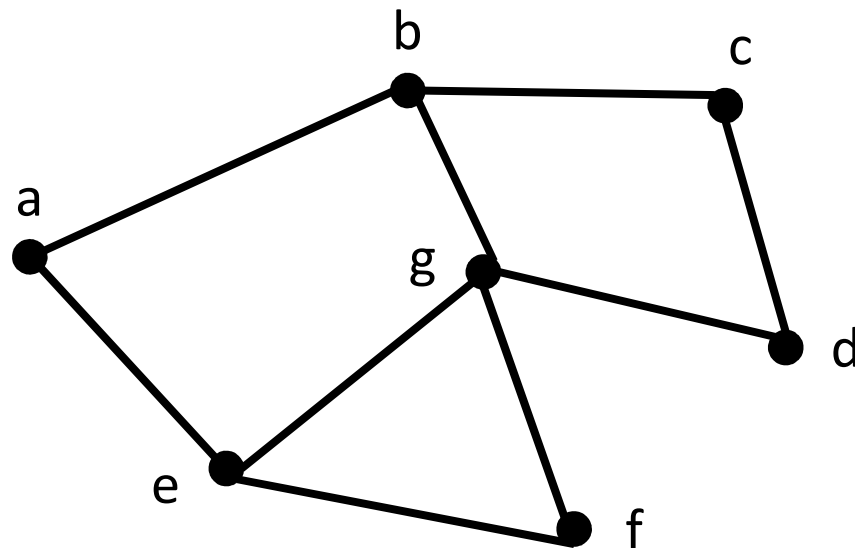
El vértice a es una raíz del dígrafo.

Arcos incidentes y grado

En un grafo no dirigido, se habla de grado porque los arcos que inciden y los que emergen son los mismos.

$\text{Grado}(v)$ = cantidad de arcos que inciden en v

Hablaremos sólo de arcos incidentes.



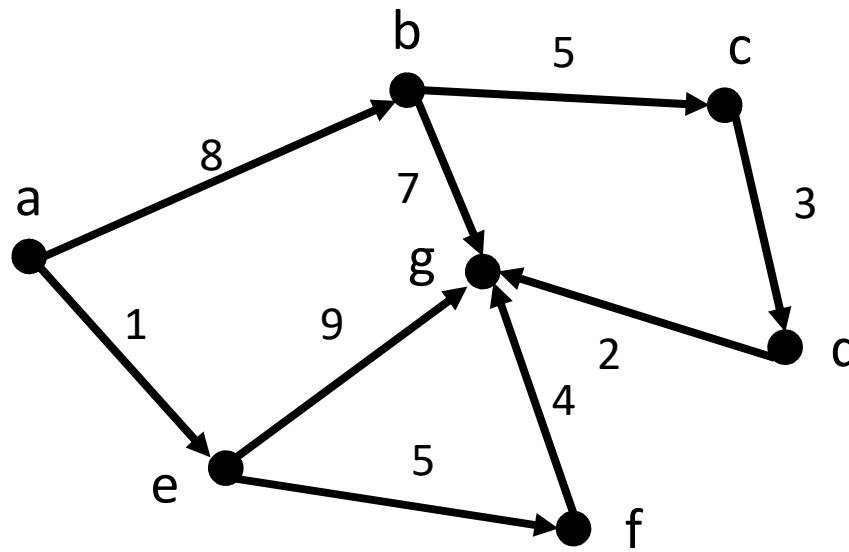
$$\text{Grado}(b) = 3$$

$$\text{Grado}(g) = 4$$

$$\text{ArcosIncidentes}(c) = \{(b,c), (c,d)\}$$

Definiciones

- Grafo pesado: Grafo donde las aristas contienen una etiqueta (o peso cuando las etiquetas son numéricas).

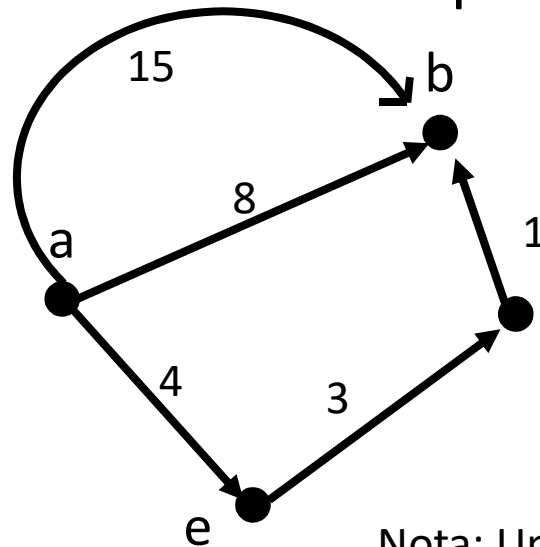


El peso de (c,d) es 3.

- Ejemplo: Digrafo donde los vértices son aeropuertos y los arcos representan vuelos (distancias entre aeropuertos si los pesos son números).

Definiciones

- Multi(di)grafo: (Di)grafo donde un par de vértices puede estar conectado por una colección de arcos. Es decir puede haber más de un arco entre cada par de vértices (se llaman “arcos paralelos”)
- Ejemplo: Digrafo donde los vértices representan aeropuertos, los arcos son vuelos y puede haber más de un vuelo entre dos aeropuertos.



Hay dos arcos que unen “a” con “b”, uno con peso 8 y otro con peso 15.

Nota: Un “grafo simple” es un grafo que no es multigrafo.

ADT Grafo

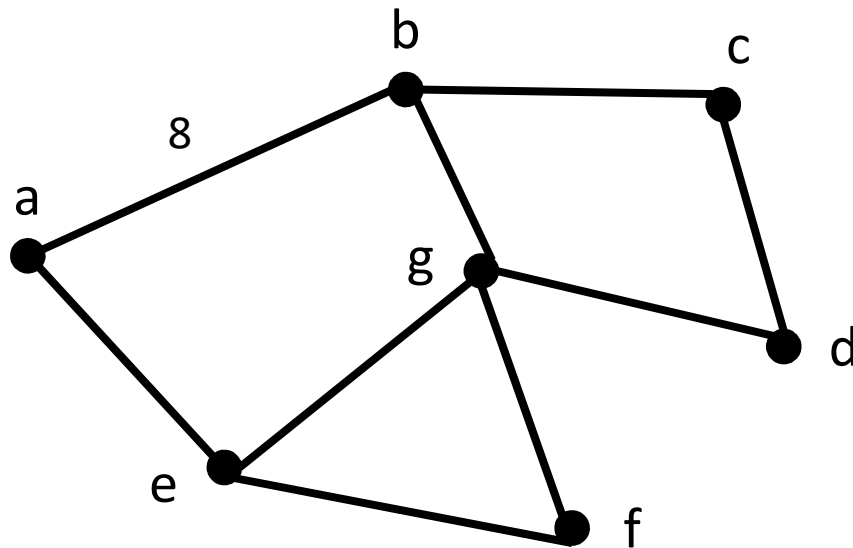
Siguiendo el libro, explicaremos primero el ADT Grafo NO dirigido:

El tipo de dato abstracto Grafo exporta tres sorts:

1. $\text{Graph}\langle V, E \rangle$: Un grafo pesado de vértices con rótulos de tipo V y arcos con rótulos de tipo E
2. $\text{Vertex}\langle V \rangle$: La posición de un vértice con rótulo de tipo V
3. $\text{Edge}\langle E \rangle$: La posición de un arco con rótulo de tipo E

ADT Grafo

- `vertices()`: Retorna una colección iterable con todos los vértices del grafo.
- `edges()`: Retorna una colección iterable con todos los arcos del grafo.



`G.vertices()` $\rightarrow$ `[va, vb, vc, vd, ve, vf, vg]`

`G.edges()` $\rightarrow$ `[eab, eae, ebc, ebg, ecd, edg, eef, eeg, efg]`

`va.element()` $\rightarrow$ `'a'`

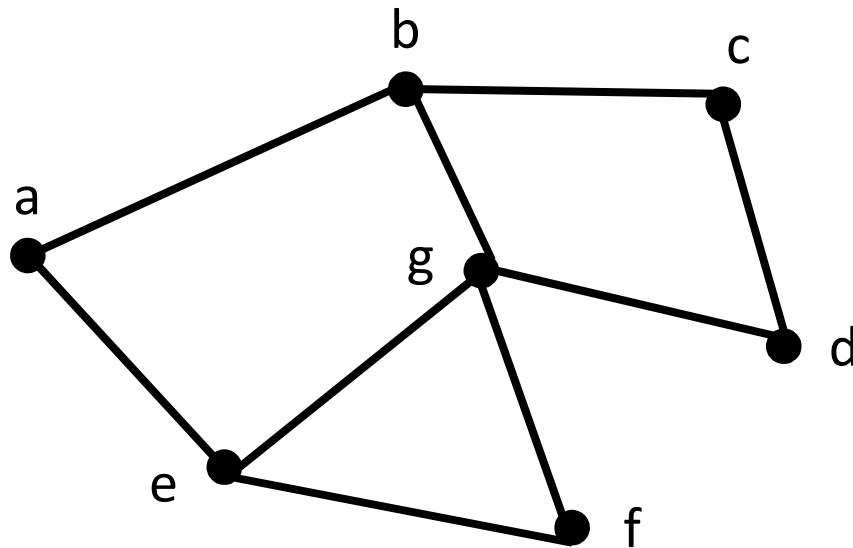
`eab.element()` $\rightarrow$ `8`

`G.vertices().iterator().next().element()` $\rightarrow$ `'a'`

`G.edges().iterator().next().element()` $\rightarrow$ `8`

ADT Grafo

`incidentEdges(v)`: Retorna una colección iterable con todos los arcos incidentes sobre un vértice v

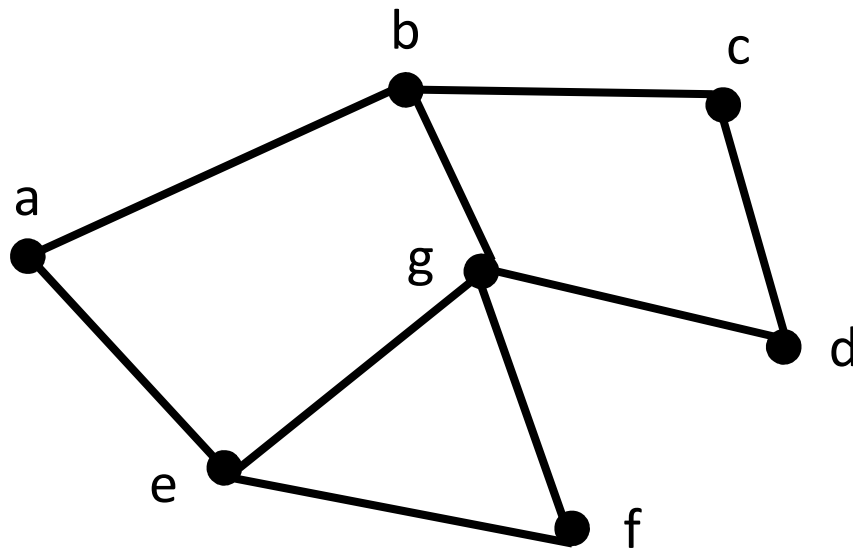


$G.\text{incidentEdges}(vg) \rightarrow$
 $[e_{bg}, e_{dg}, e_{eg}, e_{fg}]$

Notación: vg denota el vértice con rótulo g

ADT Grafo

- $\text{opposite}(v,e)$: Retorna el otro vértice w del arco $e=(v,w)$; ocurre un error si e no es incidente (o emergente de v).
- $\text{endVertices}(e)$: Retorna un arreglo (de 2 componentes) conteniendo los vértices del arco e .
- $\text{areAdjacent}(v,w)$: Testea si los vértices v y w son adyacentes.



$G.\text{opposite}(vb, e_{bg}) \rightarrow vg$

$G.\text{endVertices}(e_{bg}) \rightarrow$

vb	vg
----	----

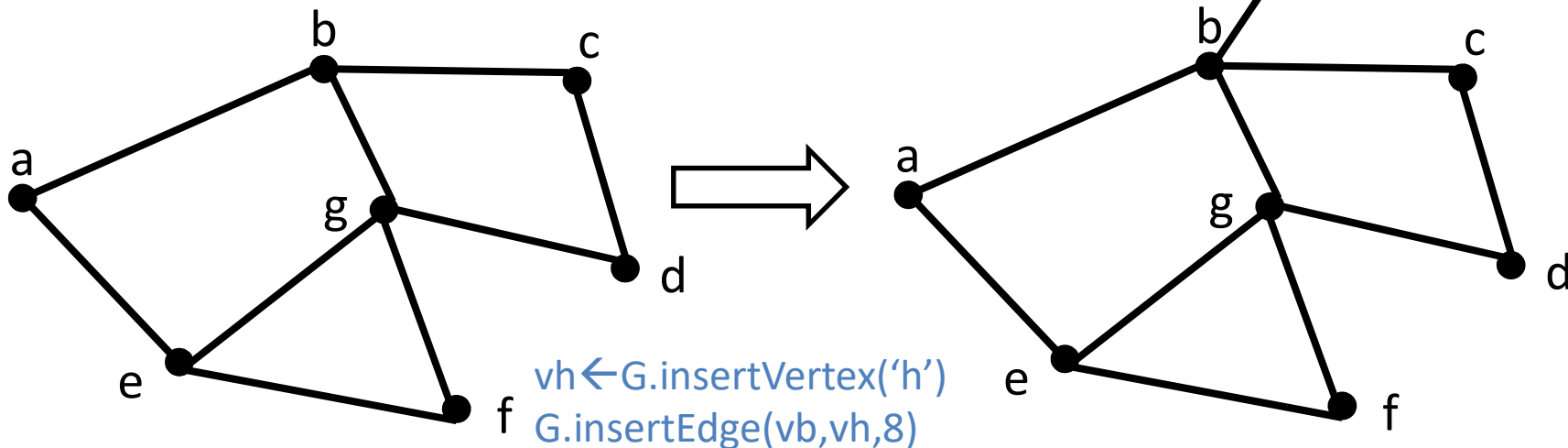
$G.\text{areAdjacent}(vb, vg) \rightarrow \text{true}$

$G.\text{areAdjacent}(ve, vc) \rightarrow \text{false}$

ADT Grafo

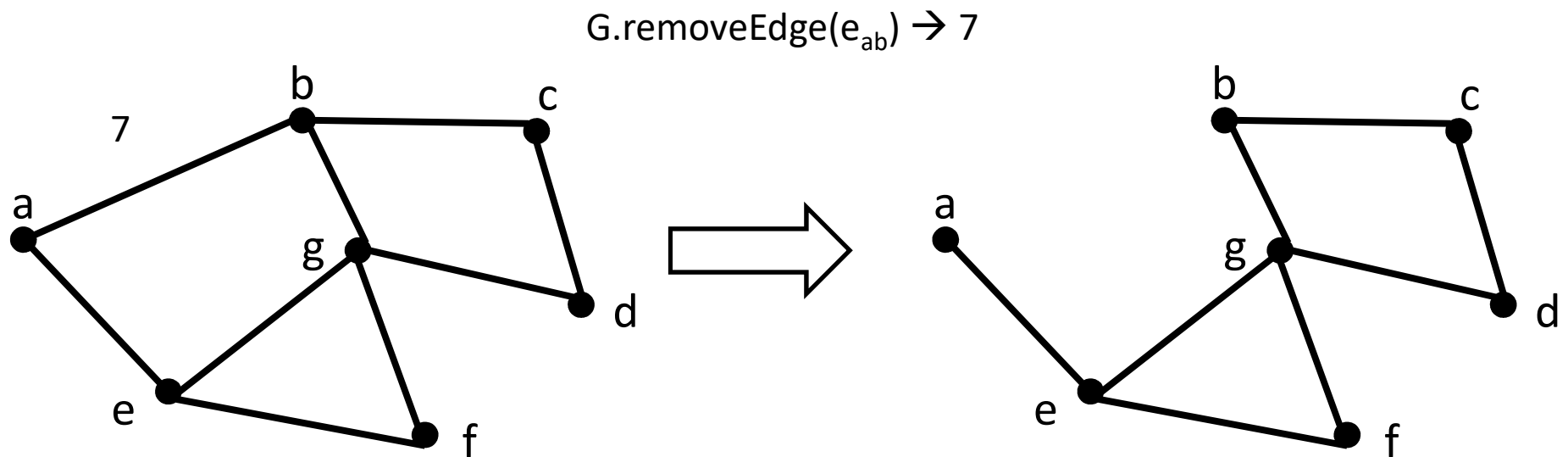
- `replace(v,x)` : Reemplaza el rótulo del vértice `v` con `x`
- `replace(e,x)`: Reemplaza el rótulo del arco `e` con `x`
- `insertVertex(x)`: Inserta y retorna un nuevo vértice con rótulo `x`
- `insertEdge(v, w, x)`: Inserta un arco con rótulo `x` entre los vértices `v` y `w`

*Insertaremos un vértice con
rótulo `h` y un arco de `b` a `h` con
rótulo `8`.*



ADT Grafo

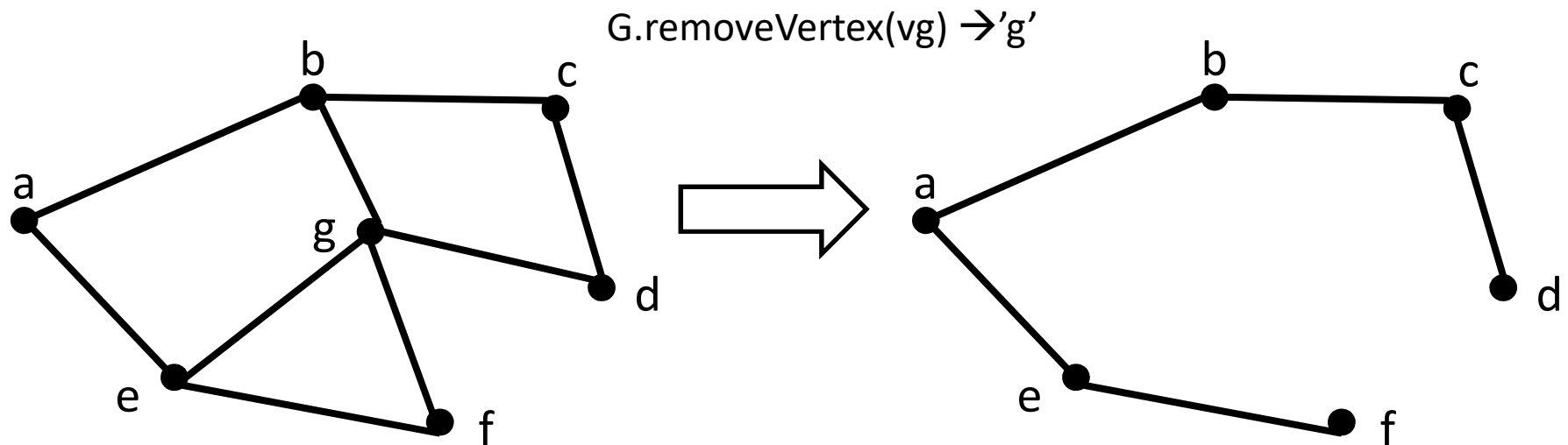
- `removeEdge(e)`: Elimina el arco e y retorna el rótulo almacenado en e .



Eliminamos el arco que conecta a con b con peso 7.

ADT Grafo

- `removeVertex(v)`: Elimina el vértice `v` y todos sus arcos adyacentes y retorna el rótulo de `v`



ADT Grafo

Creo un grafo con rótulos de vértice de tipo String
y rótulos de arco de tipo entero:

```
Graph<String, Integer> g = new Grafo<String, Integer>();
```

ADT Grafo

Creo un grafo con rótulos de vértice de tipo String y rótulos de arco de tipo entero:

```
Graph<String, Integer> g = new Grafo<String, Integer>();
```

Inserto 4 vertices: Bahia Blanca, Punta Alta, Bs As y MDP:

```
Vertex<String> bb = g.insertVertex("Bahia Blanca");
```

```
Vertex<String> pa = g.insertVertex("Punta Alta");
```

```
Vertex<String> ba = g.insertVertex("Buenos Aires");
```

```
Vertex<String> mdp = g.insertVertex("Mar del Plata");
```

Buenos Aires



Mar del
Plata



Punta Alta



Bahía
Blanca

ADT Grafo

Creo un grafo con rótulos de vértice de tipo String y rótulos de arco de tipo entero:

```
Graph<String, Integer> g = new Grafo<String, Integer>();
```

Inserto 4 vertices: Bahia Blanca, Punta Alta, Bs As y MDP:

```
Vertex<String> bb = g.insertVertex("Bahia Blanca");
```

```
Vertex<String> pa = g.insertVertex("Punta Alta");
```

```
Vertex<String> ba = g.insertVertex("Buenos Aires");
```

```
Vertex<String> mdp = g.insertVertex("Mar del Plata");
```

Inserto los arcos:

```
Edge<Integer> vueloBB2PA = g.insertEdge(bb, pa, 15);
```

Buenos Aires



Mar del
Plata

Bahía
Blanca



15km



Punta Alta

ADT Grafo

Creo un grafo con rótulos de vértice de tipo String y rótulos de arco de tipo entero:

```
Graph<String, Integer> g = new Grafo<String, Integer>();
```

Inserto 4 vertices: Bahia Blanca, Punta Alta, Bs As y MDP:

```
Vertex<String> bb = g.insertVertex("Bahia Blanca");
```

```
Vertex<String> pa = g.insertVertex("Punta Alta");
```

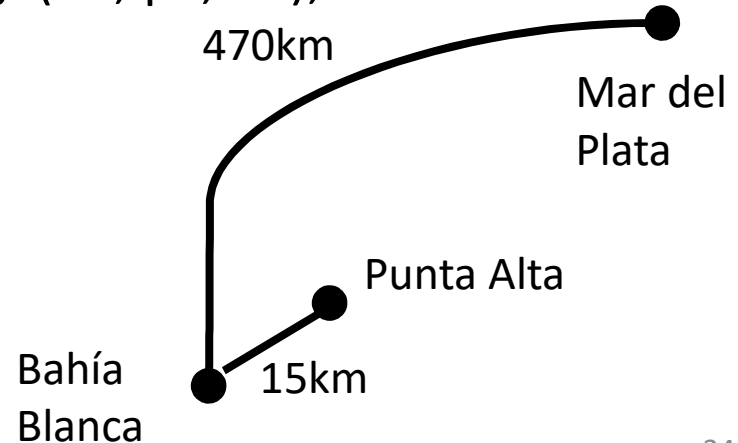
```
Vertex<String> ba = g.insertVertex("Buenos Aires");
```

```
Vertex<String> mdp = g.insertVertex("Mar del Plata");
```

Inserto los arcos:

```
Edge<Integer> vueloBB2PA = g.insertEdge(bb, pa, 15);
```

```
g.insertEdge(bb, mdp, 470);
```



ADT Grafo

Creo un grafo con rótulos de vértice de tipo String y rótulos de arco de tipo entero:

```
Graph<String, Integer> g = new Grafo<String, Integer>();
```

Inserto 4 vertices: Bahia Blanca, Punta Alta, Bs As y MDP:

```
Vertex<String> bb = g.insertVertex("Bahia Blanca");
```

```
Vertex<String> pa = g.insertVertex("Punta Alta");
```

```
Vertex<String> ba = g.insertVertex("Buenos Aires");
```

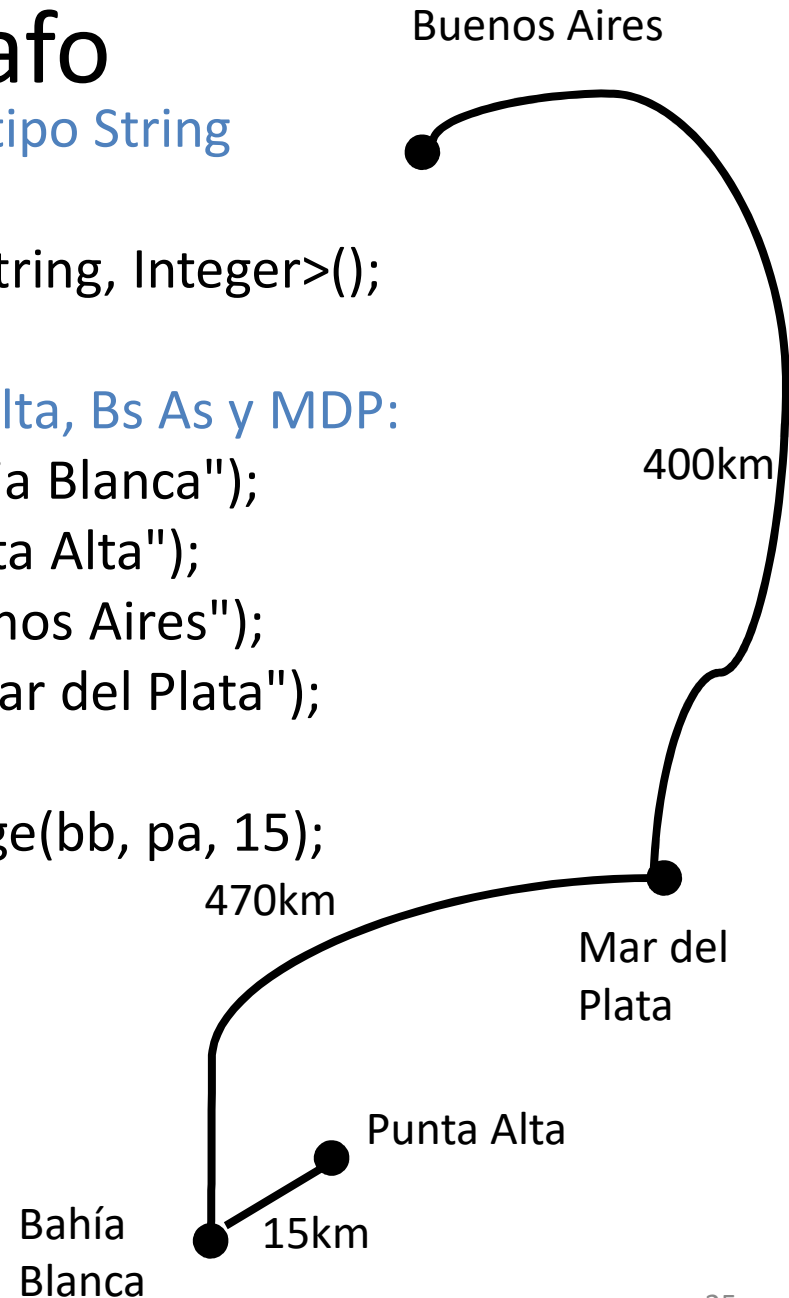
```
Vertex<String> mdp = g.insertVertex("Mar del Plata");
```

Inserto los arcos:

```
Edge<Integer> vueloBB2PA = g.insertEdge(bb, pa, 15);
```

```
g.insertEdge(bb, mdp, 470);
```

```
g.insertEdge(mdp, ba, 400);
```



ADT Grafo

Creo un grafo con rótulos de vértice de tipo String y rótulos de arco de tipo entero:

```
Graph<String, Integer> g = new Grafo<String, Integer>();
```

Inserto 4 vertices: Bahia Blanca, Punta Alta, Bs As y MDP:

```
Vertex<String> bb = g.insertVertex("Bahia Blanca");
```

```
Vertex<String> pa = g.insertVertex("Punta Alta");
```

```
Vertex<String> ba = g.insertVertex("Buenos Aires");
```

```
Vertex<String> mdp = g.insertVertex("Mar del Plata");
```

Inserto los arcos:

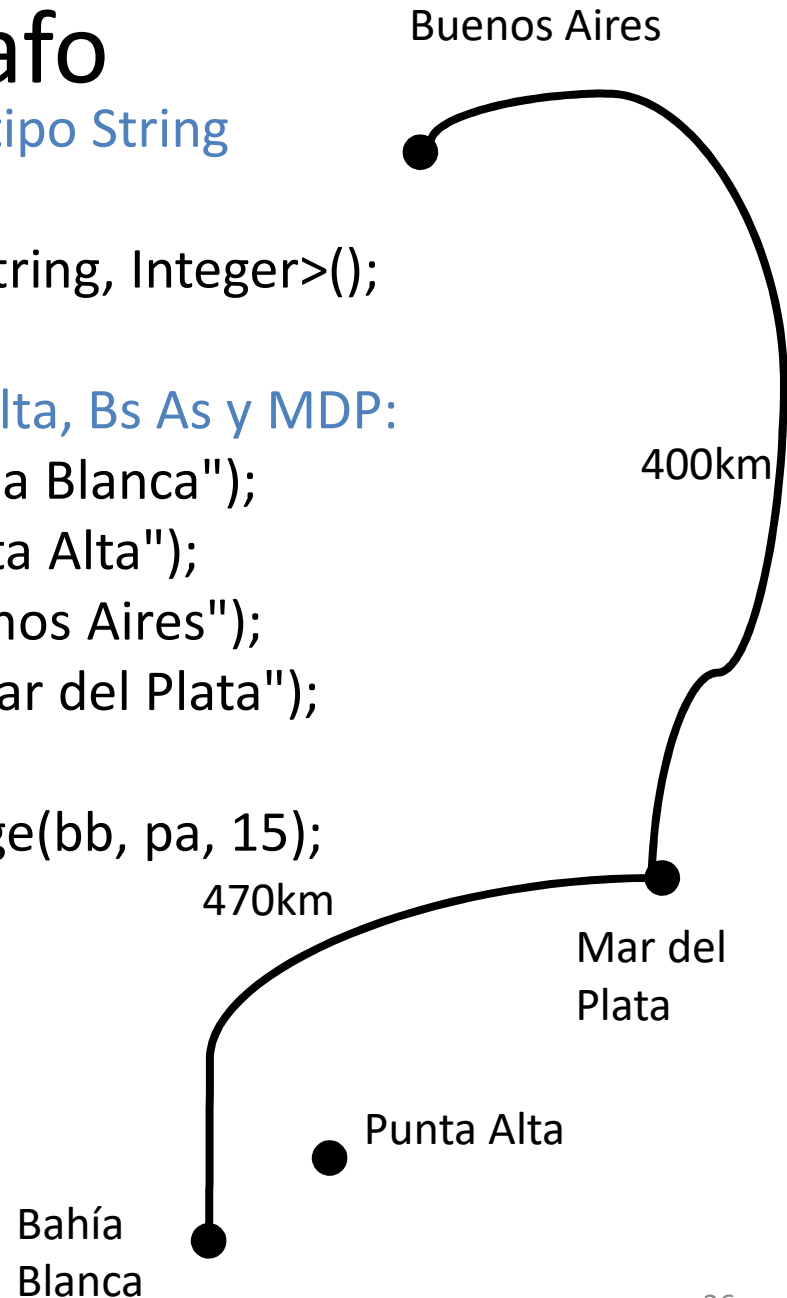
```
Edge<Integer> vueloBB2PA = g.insertEdge(bb, pa, 15);
```

```
g.insertEdge(bb, mdp, 470);
```

```
g.insertEdge(mdp, ba, 400);
```

Elimino 1 arco y 1 vértice:

```
g.removeEdge(vueloBB2PA);
```



ADT Grafo

Creo un grafo con rótulos de vértice de tipo String y rótulos de arco de tipo entero:

```
Graph<String, Integer> g = new Grafo<String, Integer>();
```

Inserto 4 vertices: Bahia Blanca, Punta Alta, Bs As y MDP:

```
Vertex<String> bb = g.insertVertex("Bahia Blanca");
```

```
Vertex<String> pa = g.insertVertex("Punta Alta");
```

```
Vertex<String> ba = g.insertVertex("Buenos Aires");
```

```
Vertex<String> mdp = g.insertVertex("Mar del Plata");
```

Inserto los arcos:

```
Edge<Integer> vueloBB2PA = g.insertEdge(bb, pa, 15);
```

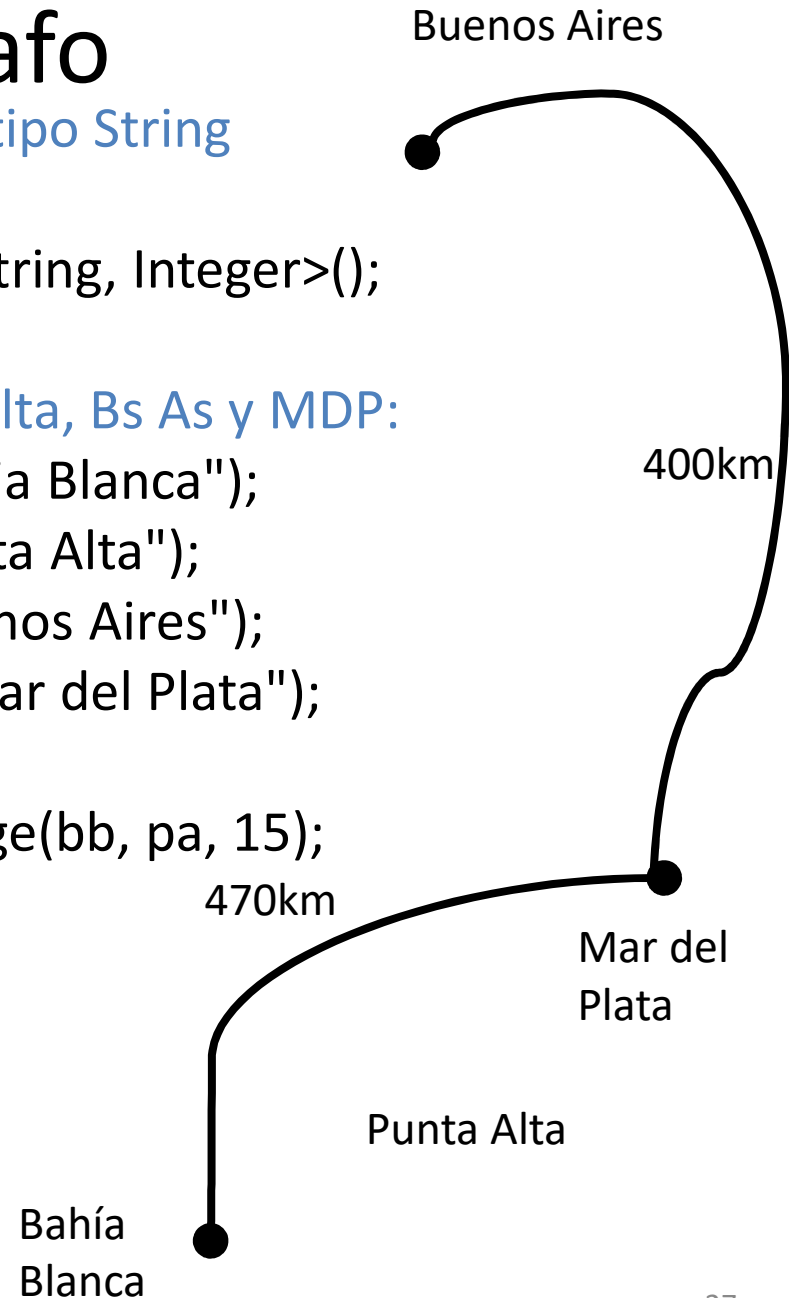
```
g.insertEdge(bb, mdp, 470);
```

```
g.insertEdge(mdp, ba, 400);
```

Elimino 1 arco y 1 vértice:

```
g.removeEdge(vueloBB2PA);
```

```
g.removeVertex(pa);
```



Estructuras de datos para grafos (simples y pesados)

- Lista de arcos: el grafo es una lista de vértices y una lista de arcos (los vértices y los arcos conocen sus rótulos respectivos).
- Lista de adyacencias: El grafo conoce una lista de vértices y una lista de arcos y cada vértice conoce su rótulo y los arcos que inciden en él. Los arcos conocen los vértices que unen y su peso.
- Matriz de adyacencias: el grafo es una lista de vértices, una lista de arcos y una matriz donde cada componente (i,j) almacena un arco.

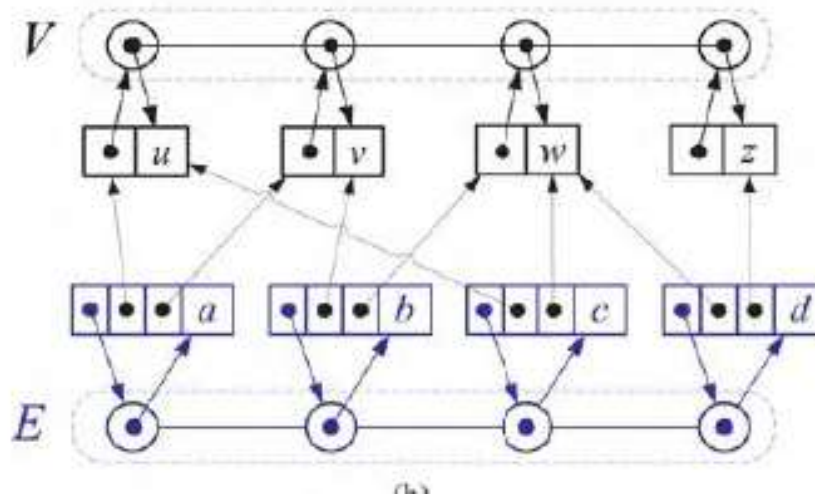
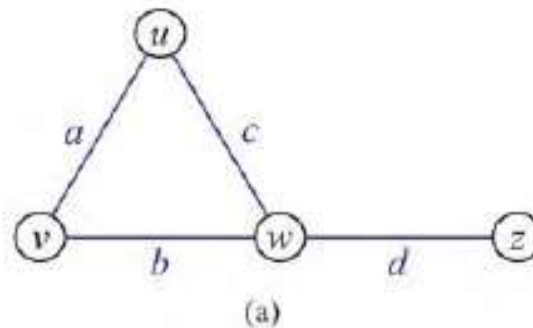
Estructuras de datos para grafos

- Lista de arcos (introdutoria para entender):
 - El grafo está representado como un par ordenado
 - La primera componente del par es una lista de vértices
 - La segunda componente del par es una lista de arcos
- Lista de adyacencias
- Matriz de adyacencias

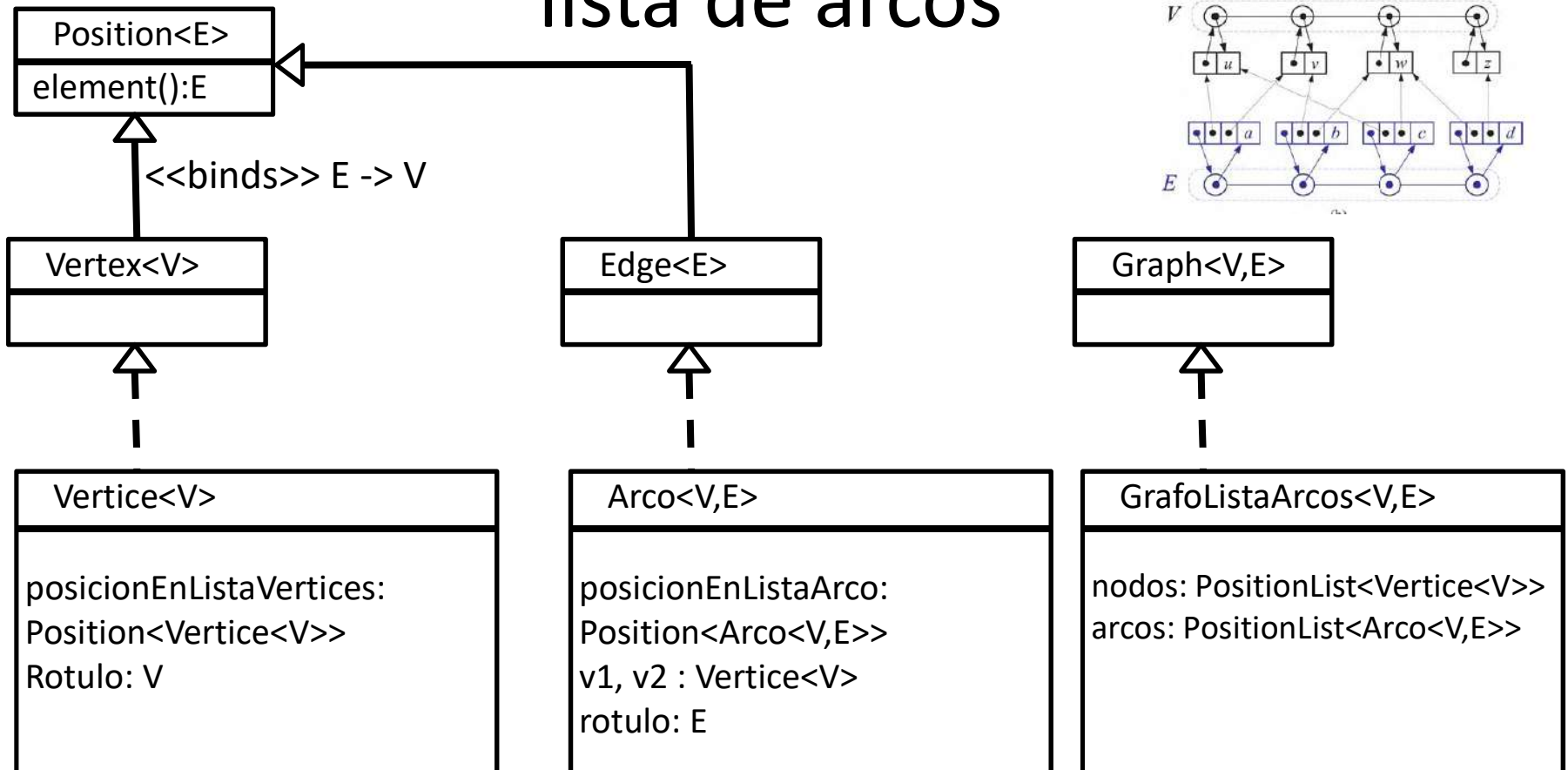
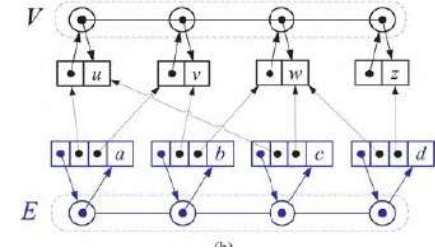
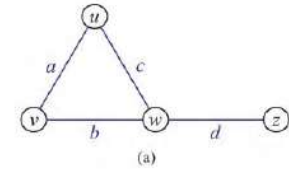
Lista de arcos: El grafo conoce una lista de vértices y una lista de arcos.

Los vértices conocen sus rótulos respectivos y su posición en la lista de vértices.

Los arcos conocen su peso, los vértices que unen y su posición en la lista de arcos.

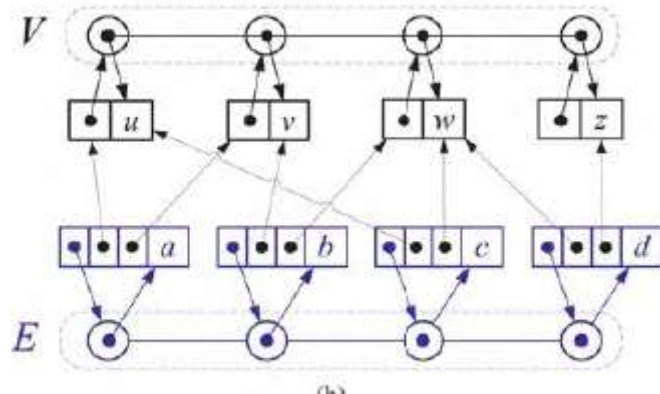
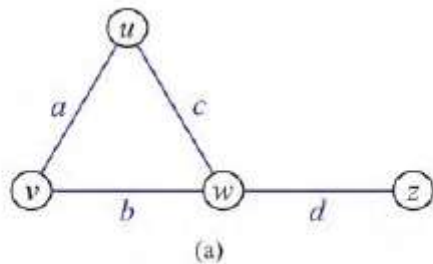


Implementación de lista de arcos



Performance de la lista de arcos

Dado un grafo $G=(V,A)$. Sea $n = \#V$ y $m = \#A$

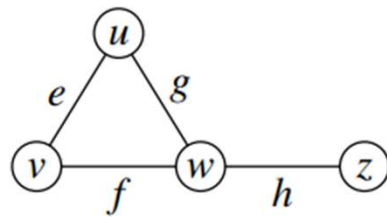


Operación	Tiempo
vertices()	$O(n)$
edges()	$O(m)$
endVertices(e), opposite(v,e)	$O(1)$
incidentEdges(v), areAdjacent(v,w)	$O(m)$
replace(v,x), replace(e,x)	$O(1)$
insertVertex(x), insertEdge(v,w,x), removeEdge(e)	$O(1)$
removeVertex(v)	$O(m)$

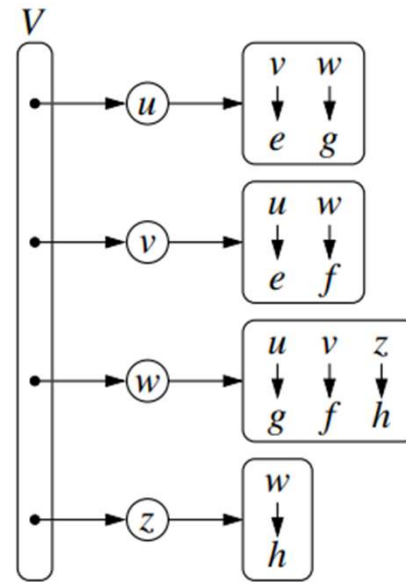
vertices() y edges() requieren copiar la lista de vertices y arcos resp. incidentEdges(v) y removeVertex(v) requieren recorrer la lista de arcos completa buscando los arcos que involucran a v.

Estructuras de datos para grafos

- Lista de adyacencias:
 - Mejora el desempeño de la lista de arcos insertando en cada vértice una lista con quiénes son sus vértices adyacentes.
 - Visión simplificada:



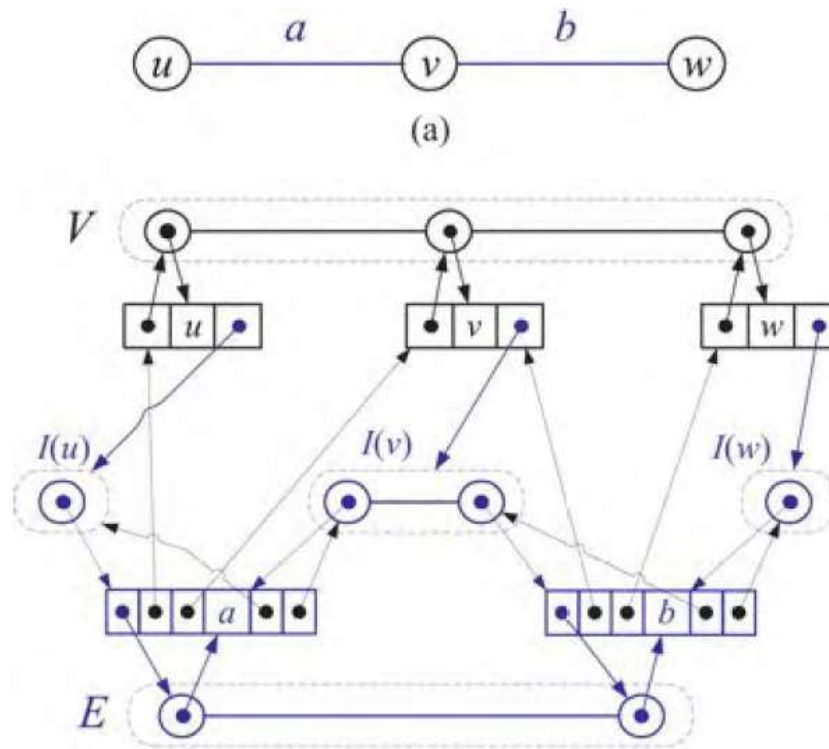
(a)



(b)

Lista de adyacencias para un grafo no dirigido $G=(V,E)$ de acuerdo a GT

La mejora que propone la lista de adyacencias es introducir un lista que para cada vértice indica los arcos que emergen cuando el grafo es dirigido y los que emergen/inciden cuando el grafo es no dirigido. El libro solo grafica los grafos no dirigidos:



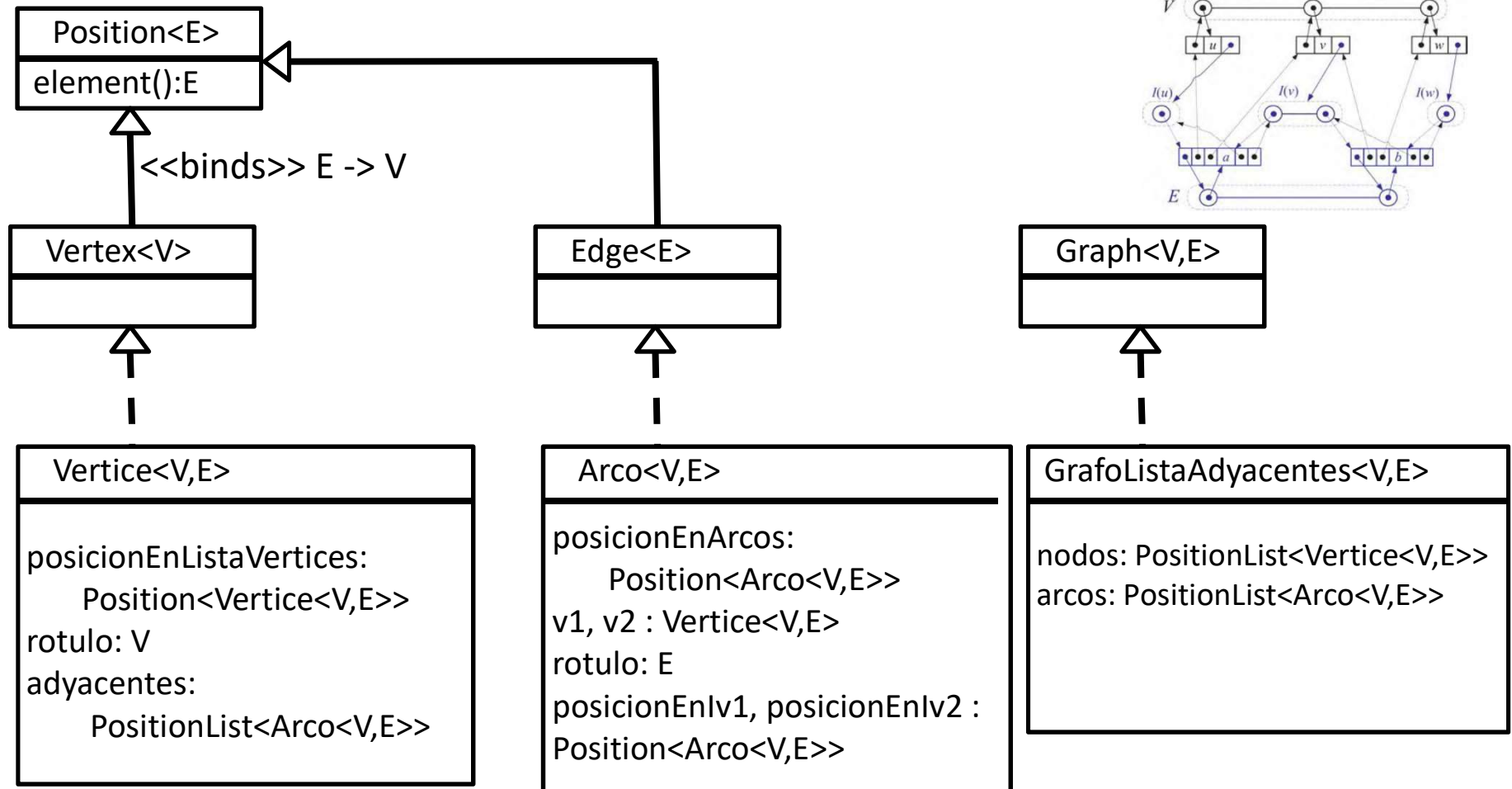
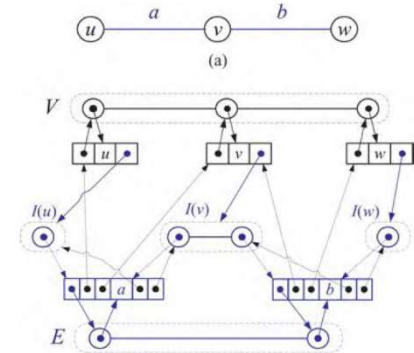
Implementación de grafo con lista de adyacencias

```
public interface Vertex<E> extends Position<E> {    } // Vertex.java (luego lo mejoraremos)
```

```
public interface Edge<E> extends Position<E> {    } // Edge.java (luego lo mejoraremos)
```

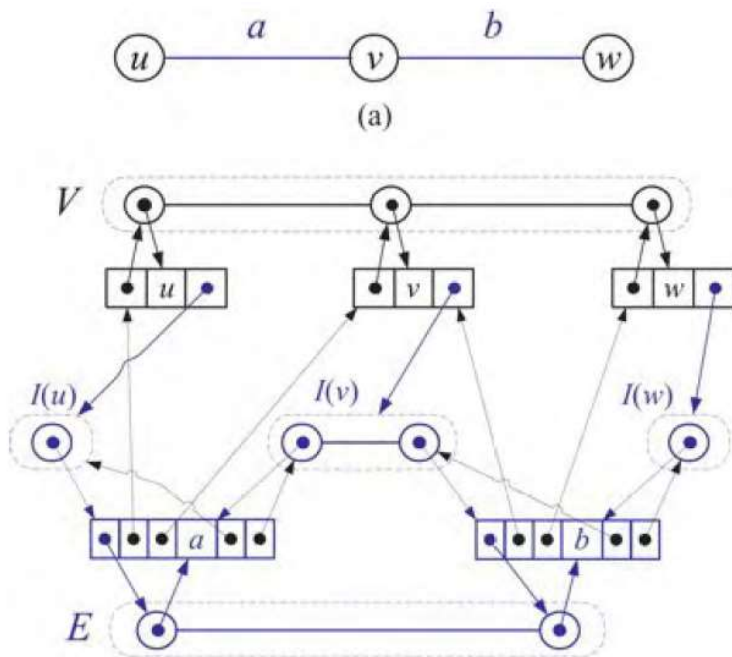
```
public interface Graph<V, E> { // Graph.java
    public Iterable<Vertex<V>> vertices();
    public Iterable<Edge<E>> edges();
    public Iterable<Edge<E>> incidentEdges(Vertex<V> v);
    public Vertex<V> opposite(Vertex<V> v, Edge<E> e);
    public Vertex<V> [] endVertices(Edge<E> e);
    public boolean areAdjacent(Vertex<V> v, Vertex<V> w);
    public V replace( Vertex<V> v, V x);
    public E replace( Edge<E> e, E x);
    public Vertex<V> insertVertex(V x);
    public Edge<E> insertEdge(Vertex<V> v, Vertex<V> w, E x);
    public V removeVertex(Vertex<V> v);
    public E removeEdge(Edge<E> e);
}
```

Implementación de grafo no dirigido con lista de adyacencias



NOTA: El libro usa la misma notación para el grafo $G=(V,E)$ y para los tipos de los rótulos de vértice V y rótulos de arco E . El campo *adyacentes* del vértice v corresponde a $I(v)$.

Performance deseada de la lista de adyacencias de acuerdo a GT



Operación	Tiempo
vertices()	$O(n)$
edges()	$O(m)$
endVertices(e), opposite(v,e)	$O(1)$
incidentEdges(v), areAdjacent(v,w)	$O(\deg(v))$
replace(v,x), replave(e,x)	$O(1)$
insertVertex(x), insertEdge(v,w,x), removeEdge(e)	$O(1)$
removeVertex(v)	$O(\deg(v))$

// Archivo: Vertice.java (o también se puede hacer clase anidada privada de la clase Grafo.java que implementa Graph)

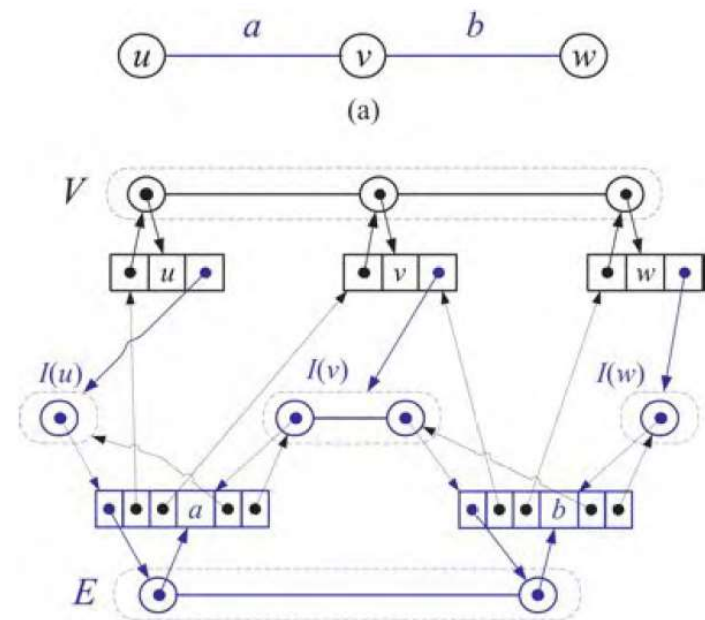
```
public class Vertice<V,E> implements Vertex<V> {
    private V rotulo;
    private PositionList<Arco<V,E>> adyacentes;
    private Position<Vertice<V,E>> posicionEnNodos;
```

```
    public V element() { return rotulo; }
```

```
    public Vertice( V rotulo ) {
        this.rotulo = rotulo;
        adyacentes = new Lista<Arco<V,E>>();
    }
```

// Setters y getters

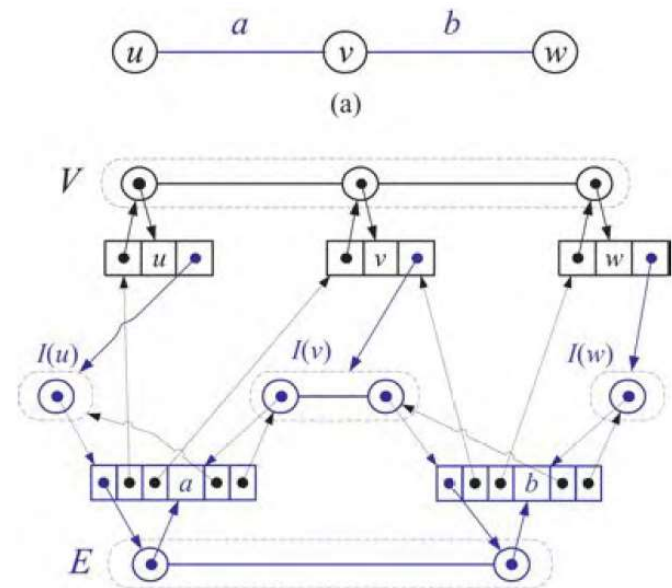
```
    public void setRotulo(V nuevoRotulo) { ... }
    public PositionList<Arco<V,E>> getAdyacentes() { ... }
    public void
        setPosicionEnNodos(Position<Vertice<V,E>> p ) { ... }
    public Position<Vertice<V,E>> getPosicionEnNodos()
    { ... }
}
```



// Archivo Arco.java: También se puede hacer clase anidada y privada de la clase Grafo que implementa Graph.

```
public class Arco<V,E>
    implements Edge<E> {
private E rotulo;
private Vertice<V,E> v1, v2;
private Position<Arco<V,E>> posicionEnArcos;
private Position<Arco<V,E>> posicionEnlv1, posicionEnlv2;
public Arco( E rotulo,
            Vertice<V,E> v1,
            Vertice<V,E> v2 )
{ /* solo setea atributos */
public E element() { return rotulo; }

// Setters y getters
}
```



// Grafo.java

```
public class Grafo<V,E> implements Graph<V,E>
{
```

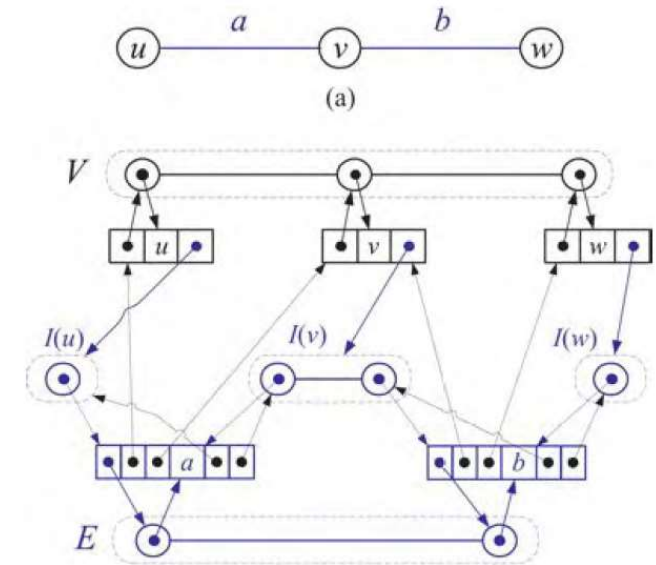
```
    protected PositionList<Vertice<V,E>> nodos;
    protected PositionList<Arco<V,E>> arcos;
```

```
public Grafo() {
    nodos = new Lista<Vertice<V,E>>();
    arcos = new Lista<Arco<V,E>>();
}
```

```
public Iterable<Vertex<V>> vertices() {
    PositionList<Vertex<V>> lista =
        new Lista<Vertex<V>>();
    for( Vertex<V> v : nodos )
        lista.addLast(v);
    return lista;
}
```

Sea n =cant de vertices y m =cantidad de arcos $\Rightarrow T_{\text{vertices}}(n, m) = O(n)$

Estructuras de datos - Dr. Sergio A. Gómez



Uso:

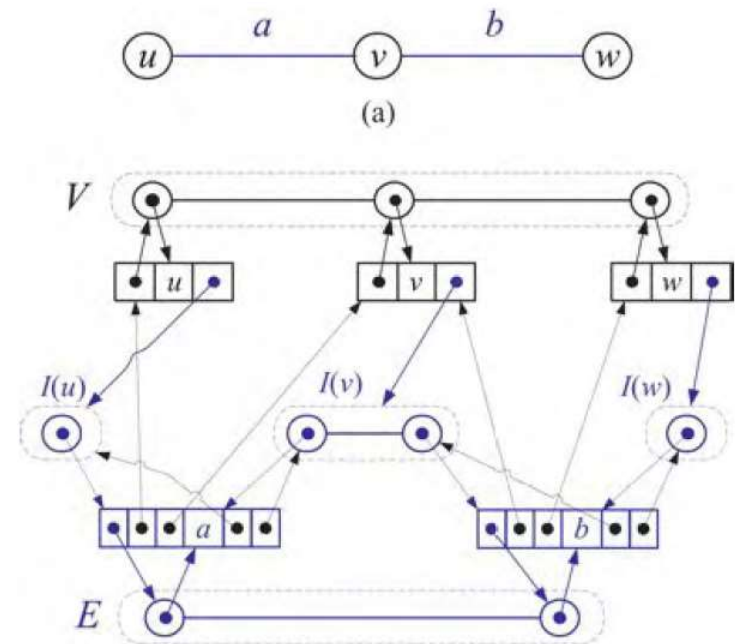
```
for( Vertex<V> v : g.vertices() )
    System.out.println( v.element() );
```


Para implementar `incidentEdges(v)` hay que recuperar la lista $I(v)$:

```
public Iterable<Edge<E>> incidentEdges( Vertex<V> v) {  
    PositionList<Edge<E>> lista = new Lista<Edge<E>>();
```

```
    Vertice<V,E> vert = (Vertice<V,E>) v;  
    for( Edge<E> e : vert.getAdyacentes() )  
        lista.addLast(e);  
    return lista;  
}
```

$T_{\text{incidentEdges}}(n,m) = O(\text{deg}(v))$



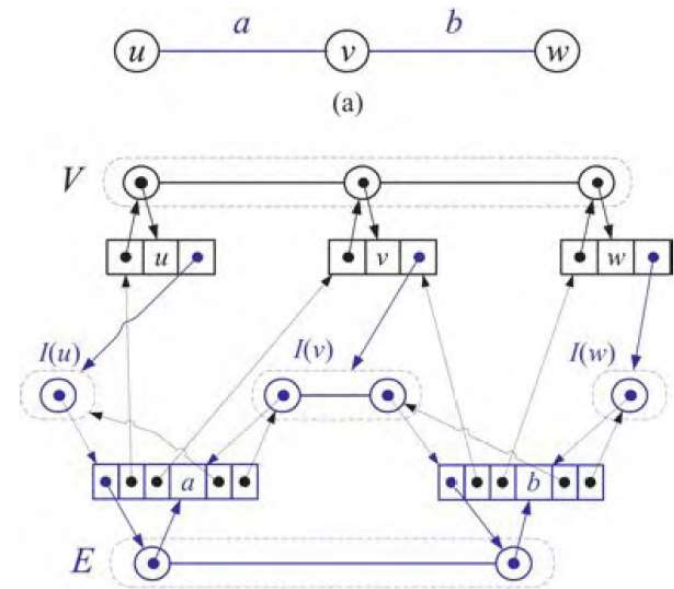
Para implementar insertVertex(x) hay que crear un vértice con rótulo x e insertarlo en la lista de vértices del grafo con una lista de adyacentes vacía:

```

public Vertex<V> insertVertex(V x) {
    Vertice<V,E> v = new Vertice<V,E>(x);
    nodos.addLast(v);
    v.setPositionEnNodos(nodos.last());
    return v;
}

```

$$T_{\text{insertVertex}}(n,m) = O(1)$$



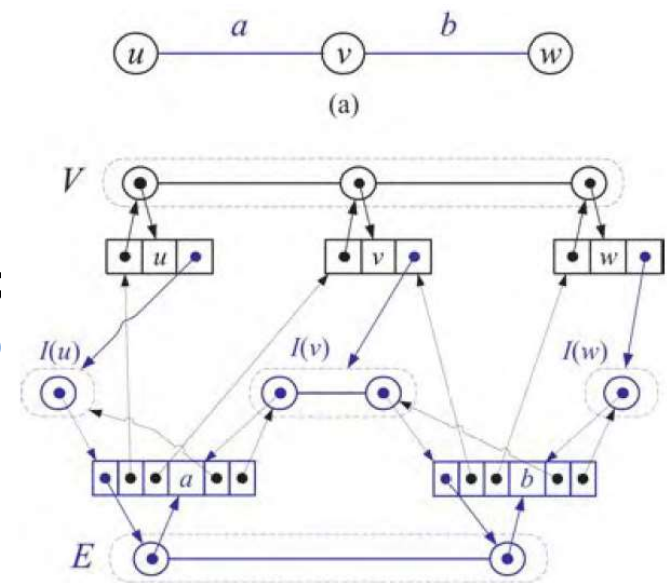
Insertar un arco requiere crear un Arco, agregarlo en la lista de arcos y en la lista de incidentes de v y en la de w . Hay que anotar en qué posición está en la lista de arcos y en qué posiciones está en la lista de adyacentes de v y en la de w .

```

public Edge<E> insertEdge(Vertex<V> v, Vertex<V> w, E x) {
    // Obtengo los vertices v y w:
    Vertice<V,E> vv = (Vertice<V,E>) v;    Vertice<V,E> ww = (Vertice<V,E>) w;
    Arco<V,E> arco = new Arco<V,E>( x, vv, ww ); // Construyo un arco
    // Agrego el arco al final de la lista de adyacentes de v y anoto su posición:
    vv.getAdyacentes().addLast( arco );
    arco.setPosicionEnlv1( vv.getAdyacentes().last() );
    // Agrego el arco al final de la lista de adyacentes
    // de w y anoto su posición:
    ww.getAdyacentes().addLast( arco );
    arco.setPosicionEnlv2( ww.getAdyacentes().last() );
    // Agrego el arco al final de lista de arcos y le seteo
    // su posición:
    arcos.addLast(arco);
    arco.setPosEnArcos(arcos.last())
    return arco; // Retorno el arco creado
}

```

$T_{\text{insertEdge}}(n,m) = O(1)$



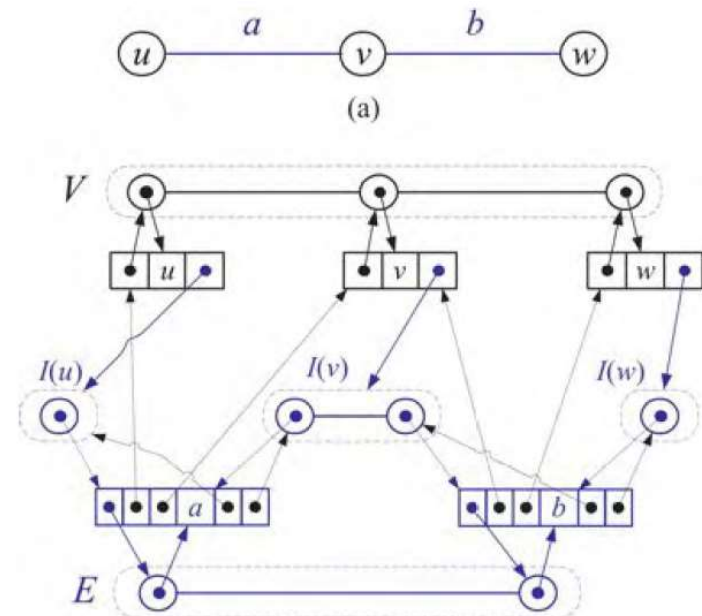
Para eliminar un arco hay que eliminarlo de las listas de incidentes de los vértices que une y luego hay que eliminarlo de la lista de arcos del grafo.

```

public E removeEdge(Edge<E> e) {
    Arco<V,E> ee = (Arco<V,E>) e; // Recupero extremos del arco:
    Vertice<V,E> v1 = ee.getV1();   Vertice<V,E> v2 = ee.getV2();
    // Elimino a e de la lista de adyacentes de v1:
    v1.getAdyacentes().remove(ee.getPosicionEnlv1());
    // Elimino a e de la lista de adyacentes de v2:
    v2.getAdyacentes().remove(ee.getPosicionEnlv2());
    // Elimino a e de la lista de arcos y retorno
    // el rótulo del arco:
    Position<Arco<V,E>> pee =
        ee.getPosicionEnArcos();
    return arcos.remove(pee).element();
}

```

$$T_{\text{removeEdge}}(n,m) = O(1)$$



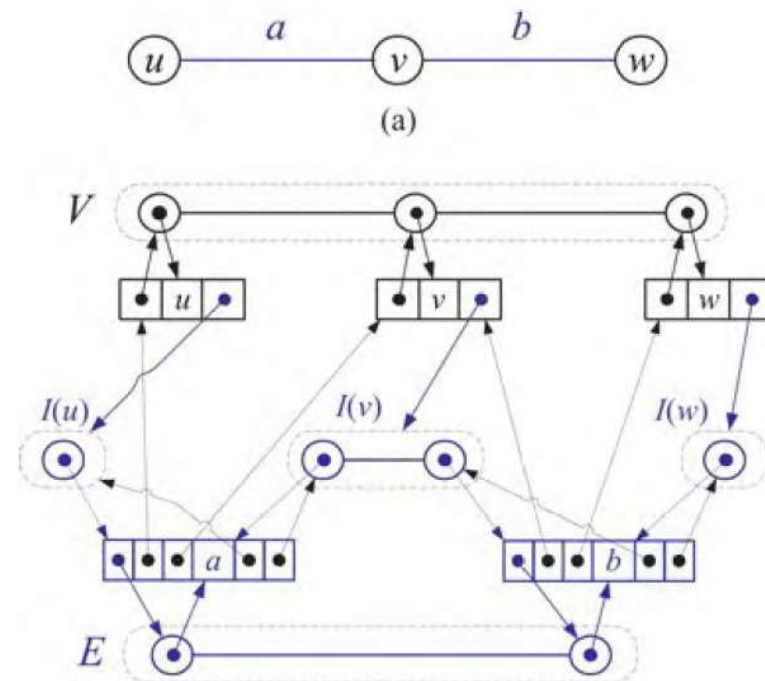
Versión simplificada de removeVertex:

// Precondición: Asume que v no tiene arcos adyacentes.

```
public V removeVertex(Vertex<V> v) {  
    Position<Vertice<V,E>> pos = ((Vertice<V,E>) v).getPosicionEnNodos();  
    return nodos.remove( pos ).element();  
}
```

$$T_{\text{removeVertex}}(n,m) = O(1)$$

Discusión: Pensar cómo modificarla para primero eliminar todos los arcos que salen del nodo v (como propone GT)



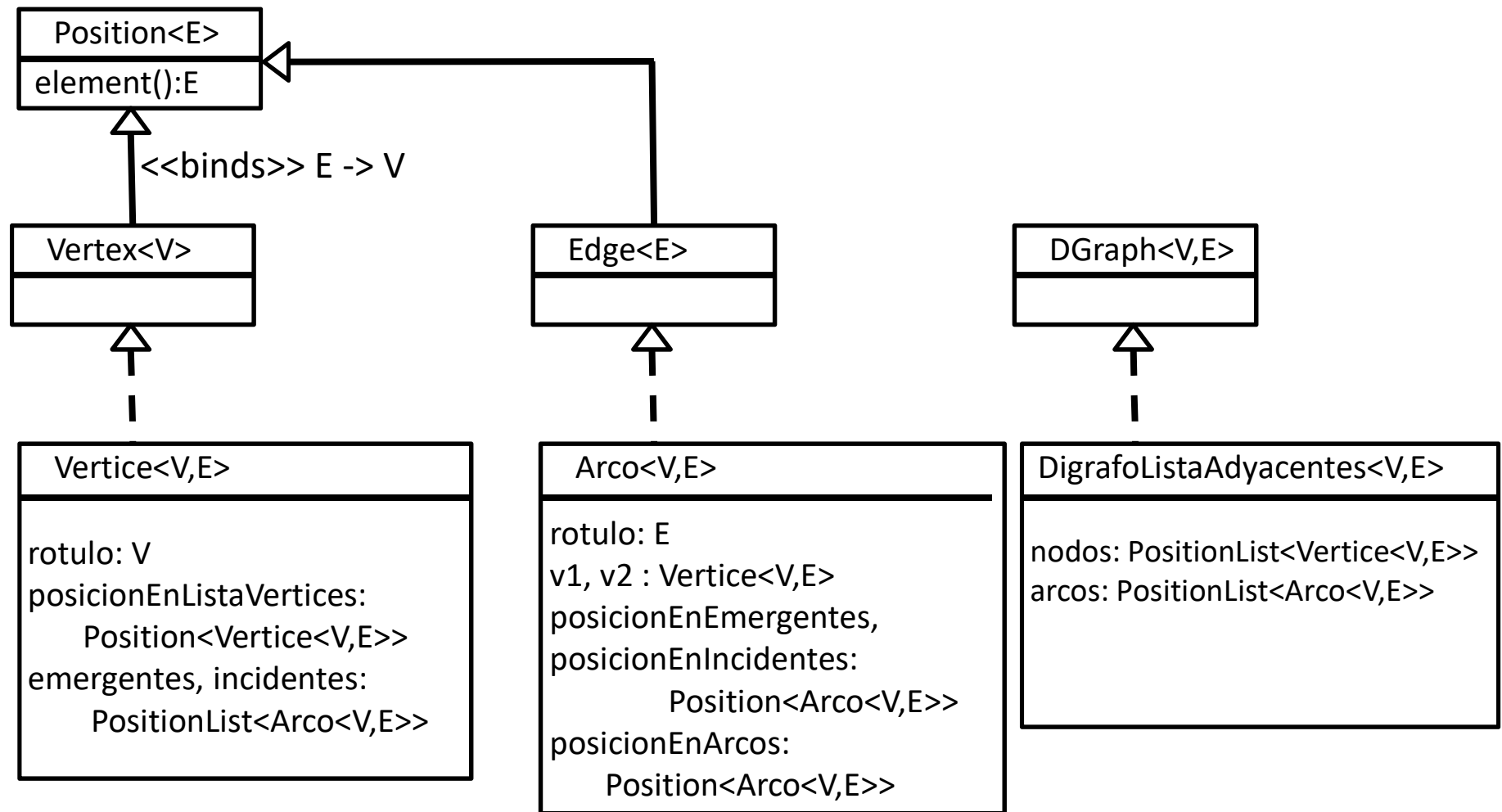
Implementación de grafos dirigidos

- Los arcos tienen dirección entonces (v,w) es diferente a (w,v)
- La operación `insertEdge(v,w,x)` inserta un arco dirigido de v a w con peso x .
- Agregamos una nueva operación `emergentEdges(v)` que retorna un iterable con los arcos que emergen de v .
- El almacenamiento de los arcos que inciden en un vértice puede o no mantenerse dependiendo de si nos interesa soportar la operación `incidentEdges(v)`.

Implementación de digrafo

```
public interface Vertex<E> extends Position<E> {    } // Vertex.java (luego lo mejoraremos)
public interface Edge<E> extends Position<E> {    } // Edge.java (luego lo mejoraremos)
public interface DGraph<V, E> {                    // DGraph.java
    public Iterable<Vertex<V>> vertices();
    // Retorna todos los arcos (dirigidos) del digrafo:
    public Iterable<Edge<E>> edges();
    // Retorna los arcos dirigidos que emergen de v:
    public Iterable<Edge<E>> emergentEdges(Vertex<V> v);
    public Iterable<Edge<E>> incidentEdges(Vertex<V> v);
    public Vertex<V> opposite(Vertex<V> v, Edge<E> e);
    public Vertex<V> [] endVertices(Edge<E> e);
    // v es adyacente a w si hay un arco dirigido de v a w:
    public boolean areAdjacent(Vertex<V> v, Vertex<V> w);
    public V replace( Vertex<V> v, V x);
    public E replace( Edge<E> e, E x);
    public Vertex<V> insertVertex(V x);
    // Inserta un arco dirigido de v a w con peso x:
    public Edge<E> insertEdge(Vertex<V> v, Vertex<V> w, E x);
    public V removeVertex(Vertex<V> v);
    public E removeEdge(Edge<E> e);
}
```

Implementación de grafo dirigido con lista de adyacencias



En el arco, v1 es la cola de flecha/arco y v2 es la punta de la flecha/arco.

v1 $\longrightarrow$ v2

Estructuras de datos para grafos

- Lista de arcos
- Lista de adyacencias
- Matriz de adyacencias:
 - Reemplazo las listas de adyacencias por una única matriz donde la componente (i,j) es el arco que conecta el vértice i con el vértice j .

Grafo con Matriz

```
Graph<String, Integer> g = new GrafoConMatriz<String, Integer>();
```

```
Vertex<String> bb = g.insertVertex("Bahia Blanca");
```

```
Vertex<String> pa = g.insertVertex("Punta Alta");
```

```
Vertex<String> ba = g.insertVertex("Buenos Aires");
```

```
Vertex<String> mdp = g.insertVertex("Mar del Plata");
```

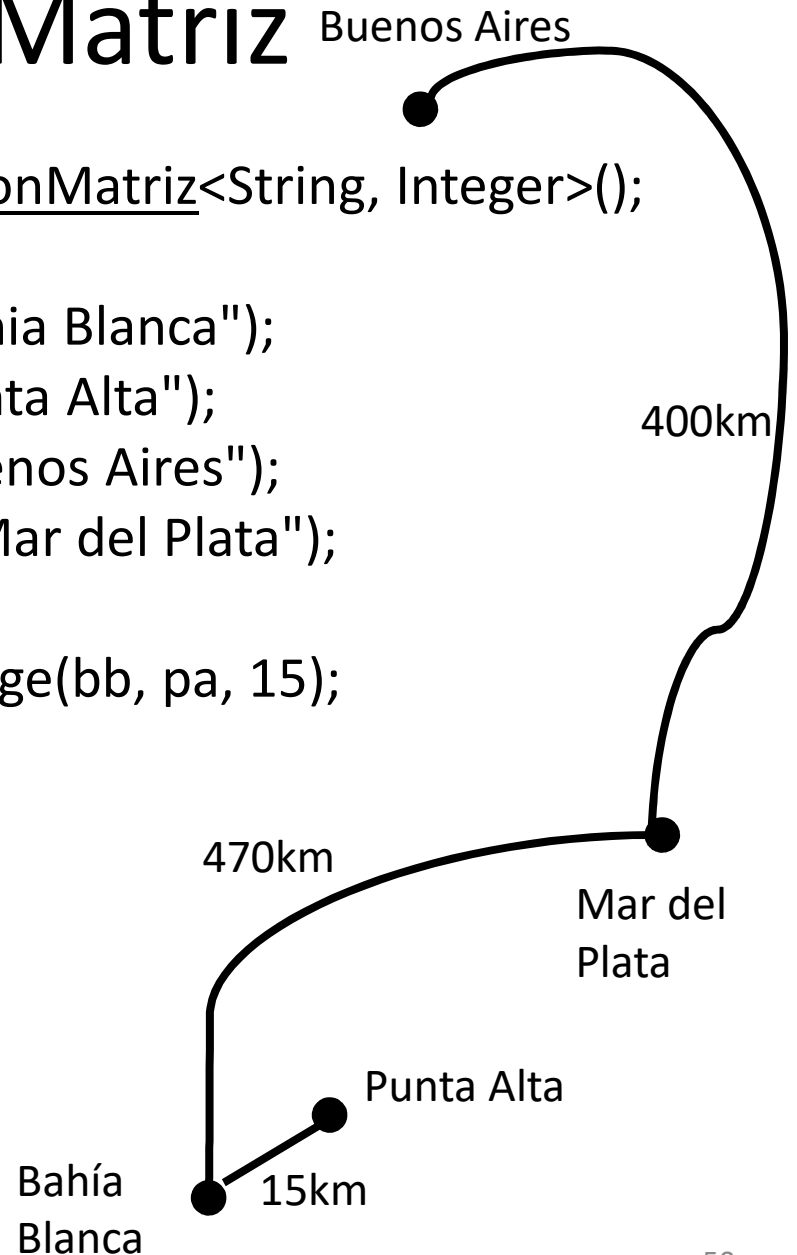
```
Edge<Integer> vueloBB2PA = g.insertEdge(bb, pa, 15);
```

```
g.insertEdge(bb, mdp, 470);
```

```
g.insertEdge(mdp, ba, 400);
```

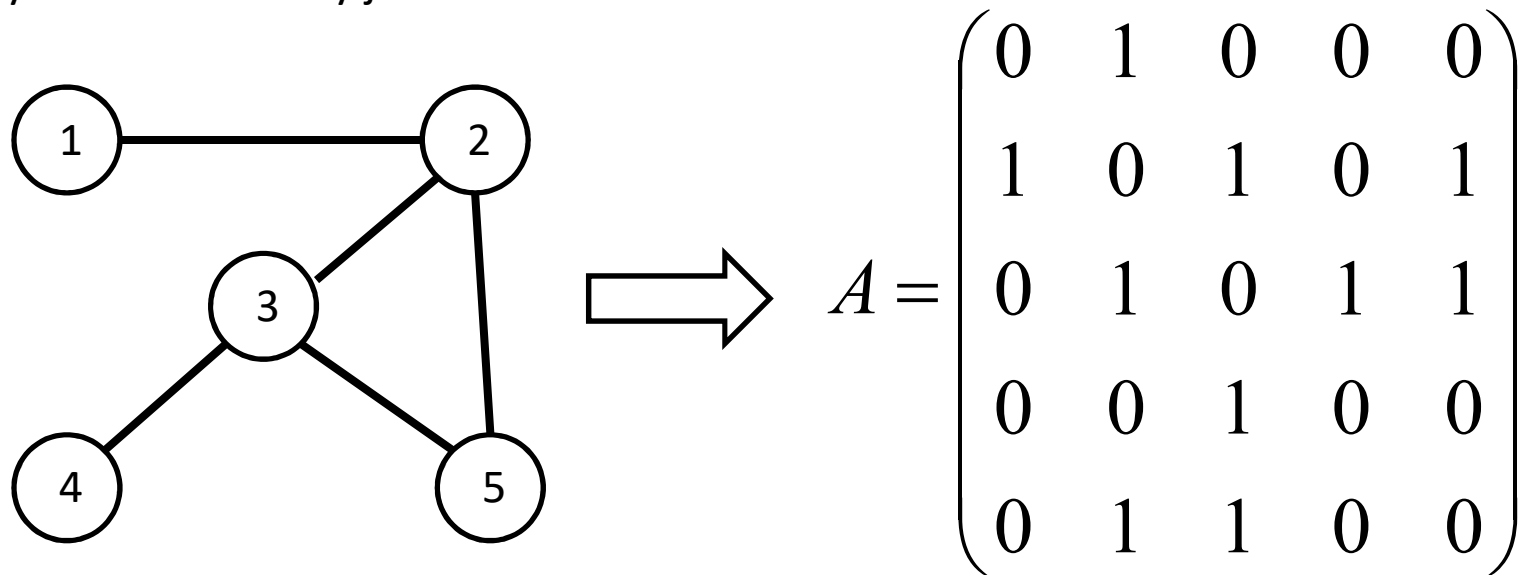
```
g.removeEdge(vueloBB2PA);
```

```
g.removeVertex(pa);
```



Enfoques matemáticos para representar un grafo con una matriz de adyacencias

Un grafo G no dirigido con vértices numéricos se puede representar matemáticamente por una matriz booleana $A=(a_{ij})$ con $i,j=1,\dots,n$ donde $a_{ij} = 1$ quiere decir que hay un arco entre los vértices i y j y $a_{ij} = 0$ quiere decir que no hay un arco entre i y j .



Sea $G=(V,E)$ donde:

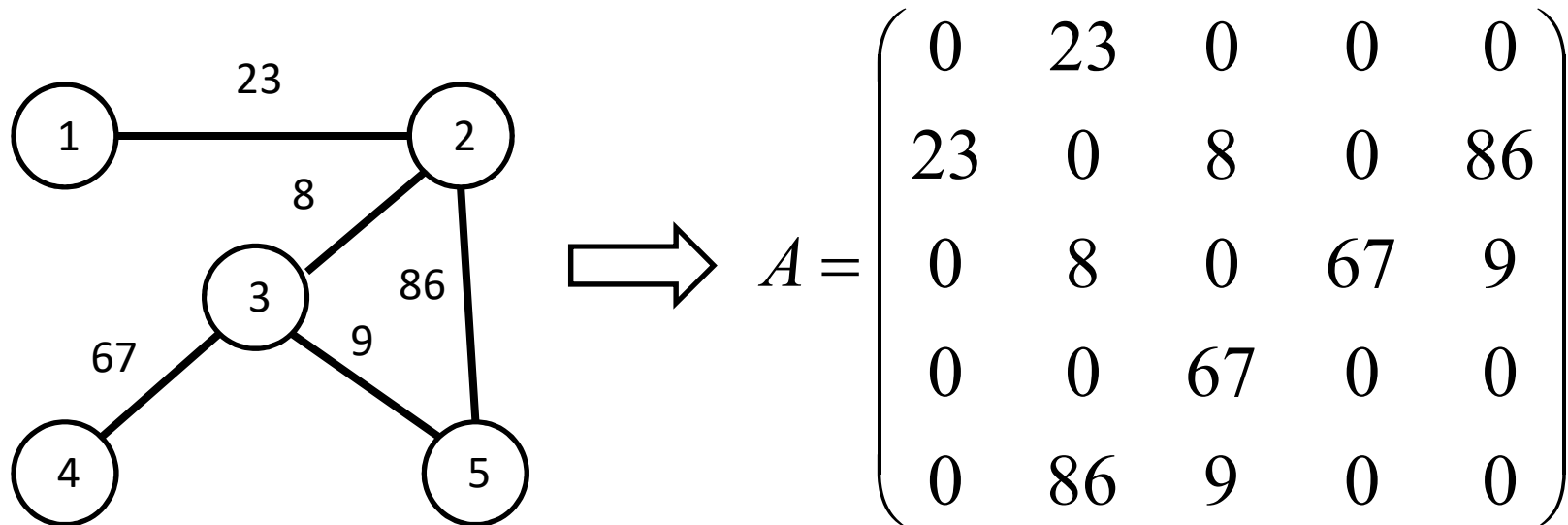
$V=\{1,2,3,4,5\}$

$E = \{ (1,2), (2,3), (2,5), (3,4), (3,5) \}$

Por ejemplo, como $(2,3)$ es un arco entonces $a_{23} = a_{32} = 1$.

Enfoques matemáticos para representar un grafo pesado con una matriz de adyacencias

Un grafo G no dirigido pesado con vértices numéricos se puede representar matemáticamente por una matriz real $A=(a_{ij})$ con $i,j=1,\dots,n$ donde a_{ij} es el peso del arco.



Sea $G=(V,E)$ donde:

$V=\{1,2,3,4,5\}$

$E = \{ (1,2), (2,3), (2,5), (3,4), (3,5) \}$

$w(1,2)=23, w(2,3)=8, w(2,5)=86,$

$w(3,4)=67, w(3,5)=9.$

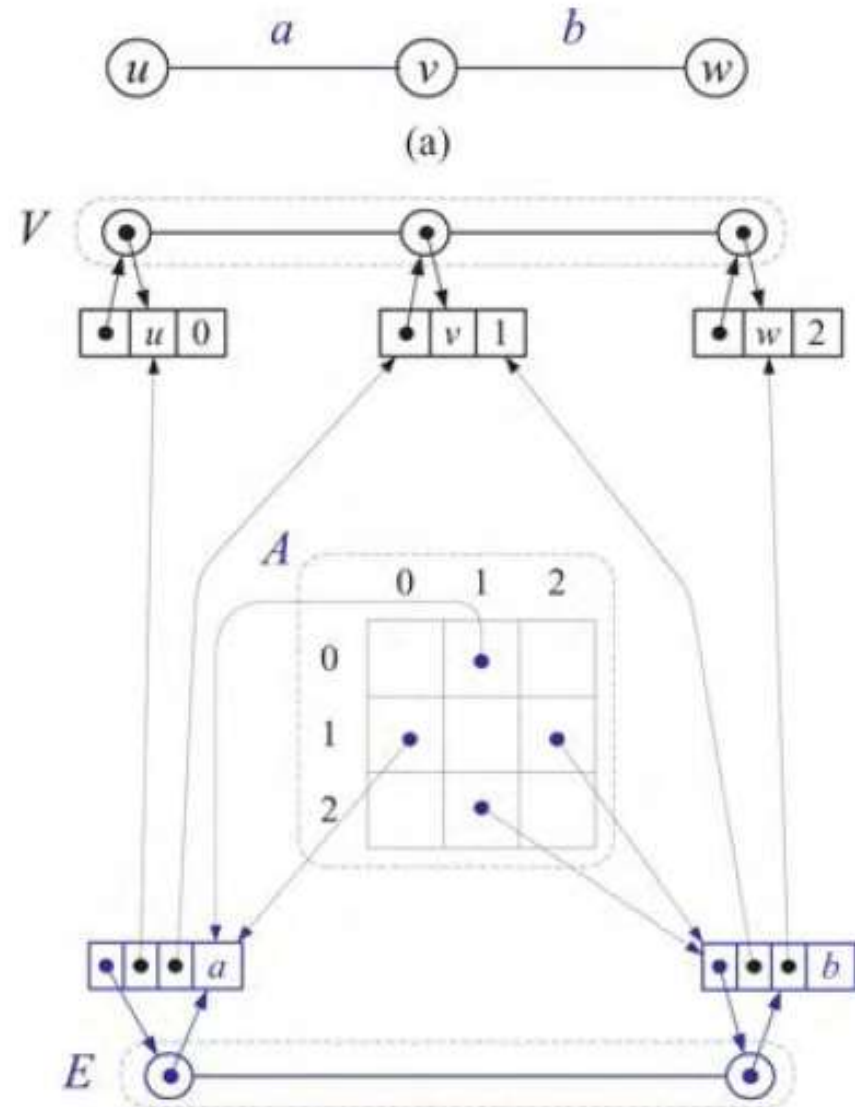
Por ejemplo, como $(2,3)$ es un arco con peso 8 (notado como $w(2,3)=8$) entonces $a_{23} = a_{32} = 8$.

Matriz de adyacencias

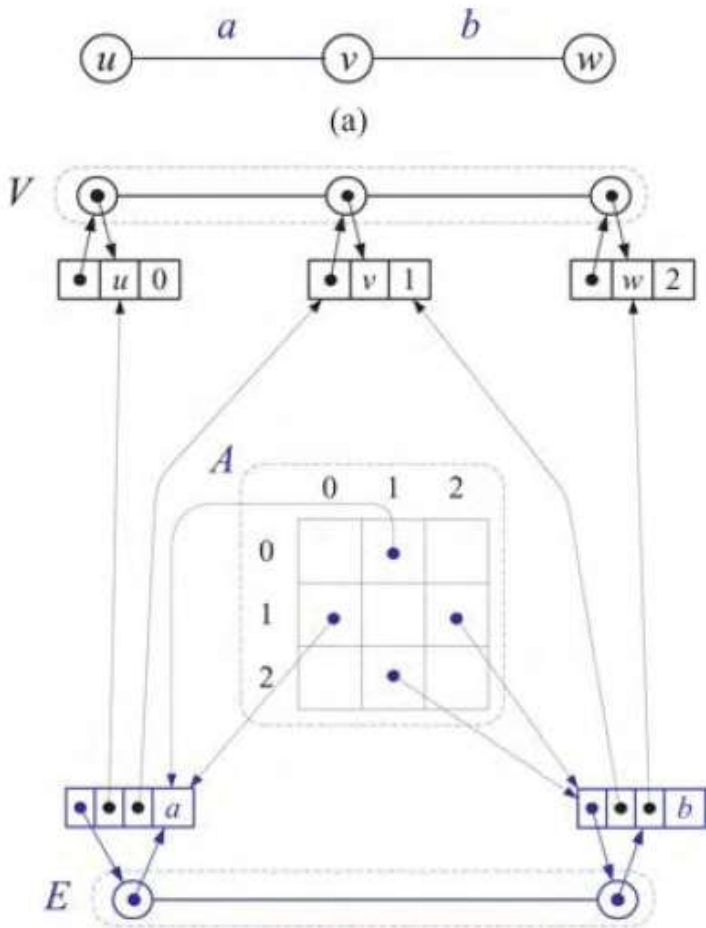
En la representación de matriz de adyacencias además de la matriz de arcos tendremos una lista de vértices y una lista de arcos.

Cada vértice conoce su posición en la lista de vértices, el rótulo del vértice y su *índice* en la matriz de arcos.
Como usamos Java, si tenemos n vértices, los índices de vértices van de 0 a $n-1$.

Cada arco conoce su posición en la lista de arcos, los vértices que conecta y el peso del arco.



Performance de matriz de adyacencias



Operación	Tiempo
vertices()	$O(n)$
edges()	$O(m)$
endVertices(e), opposite(v,e), areAdjacent(v,w)	$O(1)$
incidentEdges(v)	$O(n + \deg(v))$
replace(v,x), replace(e,x), insertEdge(v,w,x), removeEdge(e)	$O(1)$
insertVertex, removeVertex	$O(n^2)$

¿Por qué insertVertex y removeVertex tienen $O(n^2)$?

¿Cuál es el $T(n)$ de `removeVertex(v)` si asumimos que no hay arcos que salen o llegan a v ?

En `removeVertex(v)`, ¿qué podría hacer para tener $O(1)$ si v corresponde a una fila en el medio de la matriz?

Implementación de grafo no dirigido con matriz de adyacencias

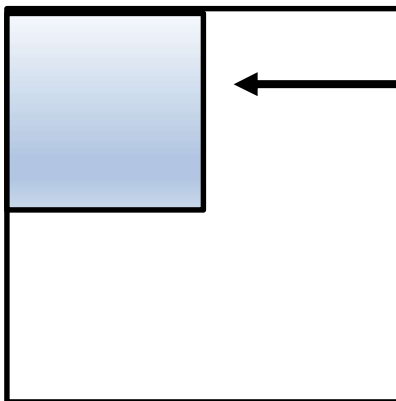
```
public class GrafoConMatriz<V,E> implements Graph<V,E> {
```

```
    protected PositionList<Vertex<V>> vertices;
```

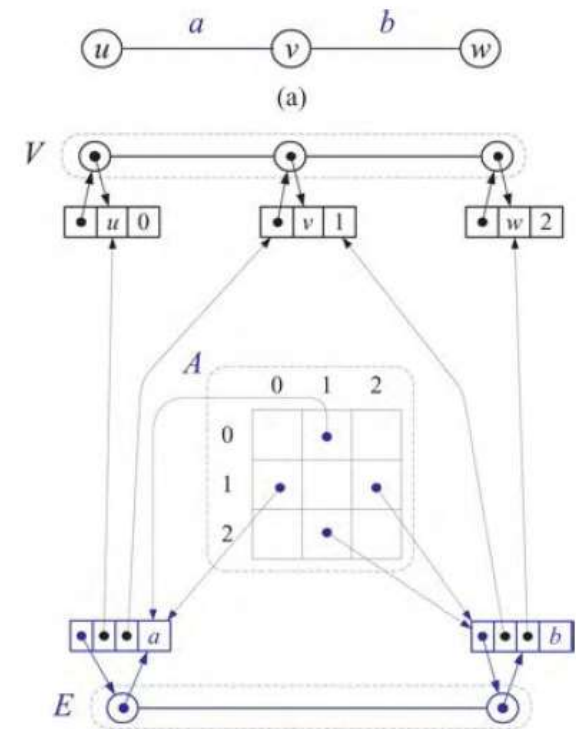
```
    protected PositionList<Edge<E>> arcos;
```

```
    protected Edge<E> [][] matriz;
```

```
    protected int cantidadVertices;
```



Usaremos una porción de cantidadVertices por cantidadVertices de la matriz.



```
//      protected PositionList<Vertex<V>> vertices;
//      protected PositionList<Edge<E>> arcos;
//      protected Edge<E> [][] matriz;
//      protected int cantidadVertices;
```

```
private class Vertice<V> implements Vertex<V> {
    private Position<Vertex<V>> posicionEnVertices;
    private V rotulo;
    private int indice;
```

```
    public Vertice( V rotulo, int indice ) { ... setear campos rotulo e índice... }
```

```
    // setters y getters:
```

```
    public void setPosicionEnVertices( Position<Vertex<V>> p ) { ... }
```

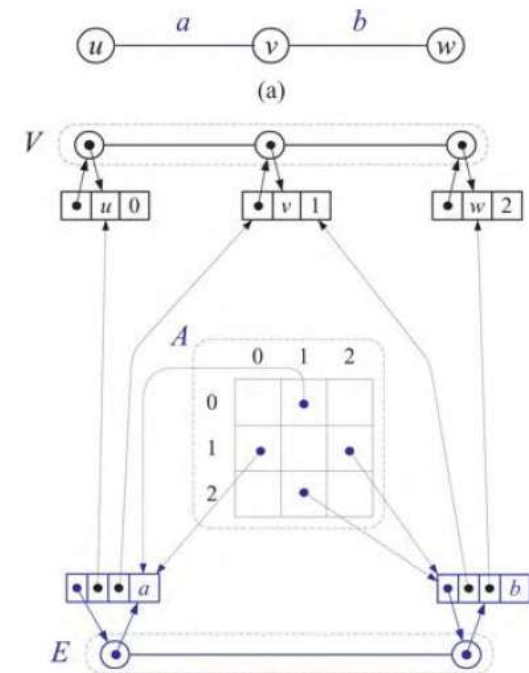
```
    public void setRotulo(V nuevoRotulo) { ... }
```

```
    public int getIndice() { ... }
```

```
    public Position<Vertex<V>> getPositionEnVertices() { ... }
```

```
    public V element() { ... }
```

```
}
```



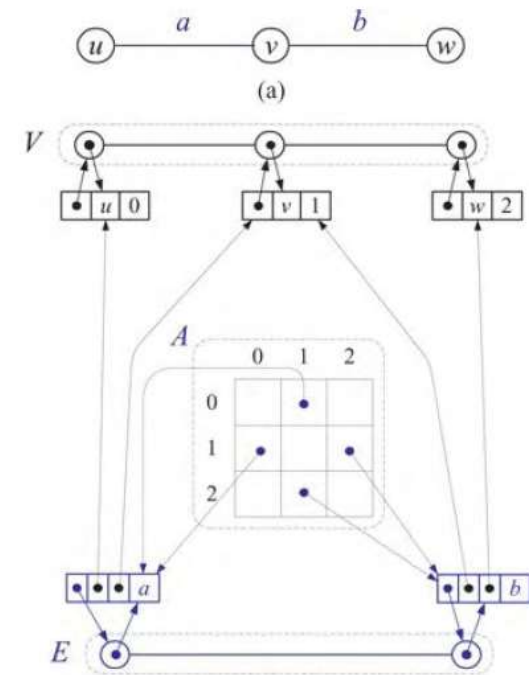

```
//      protected PositionList<Vertex<V>> vertices;
//      protected PositionList<Edge<E>> arcos;
//      protected Edge<E> [][] matriz;
//      protected int cantidadVertices;
```

```
private class Arco<V,E> implements Edge<E> {
private Position<Edge<E>> posicionEnArcos;
private Vertice<V> v1, v2;
private E rotulo;
```

```
public Arco( E rotulo, Vertice<V> v1, Vertice<V> v2 ) {
    ...setear campos del arco... }
```

// setters y getters:

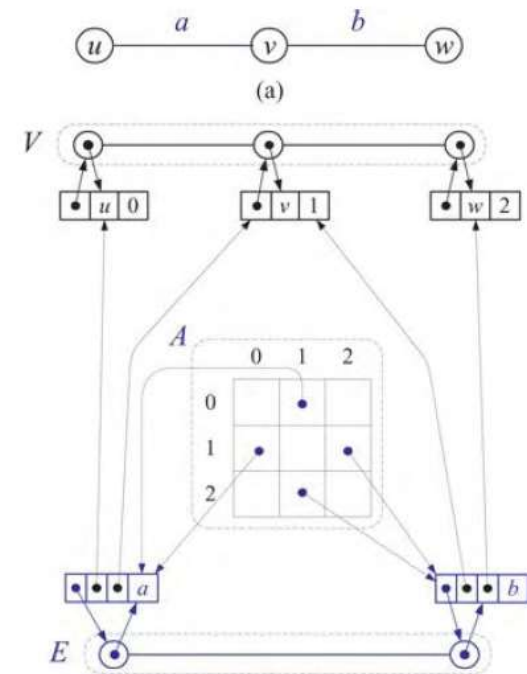
```
public void setPosicionEnArcos( Position<Edge<E>> p ) { ... }
public E element() { ... }
public Position<Edge<E>> getPosicionEnArcos() { ... }
public Vertice<V> getV1() { ... }
public Vertice<V> getV2() { ... }
public void setRotulo(E nuevoRotulo) { ... }
}
```



```
//      protected PositionList<Vertex<V>> vertices;
//      protected PositionList<Edge<E>> arcos;
//      protected Edge<E> [][] matriz;
//      protected int cantidadVertices;
```

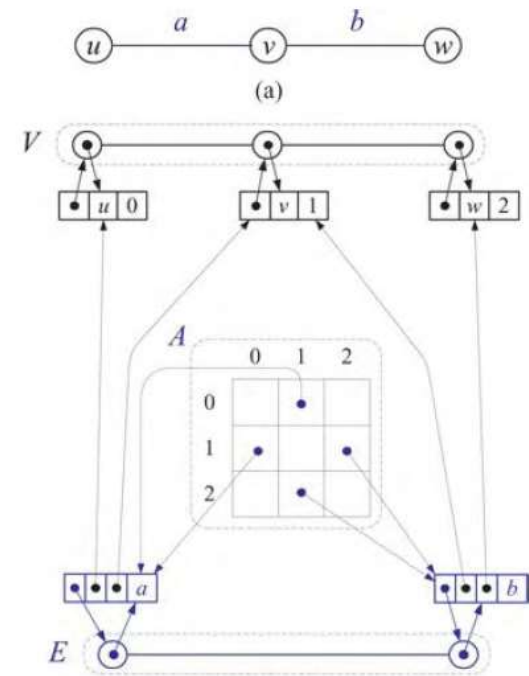
```
public GrafoConMatriz( int n ){ // Recibe el tamaño de la matriz
    vertices = new DoubleLinkedList<Vertex<V>>();
    arcos = new DoubleLinkedList<Edge<E>>();
    // Produce warning
    // unsafe operation: compilar con javac -Xlint:unchecked
    // GrafoConMatriz.java
    matriz = (Edge<E> [][]) new Arco[n][n];
    cantidadVertices = 0;

    for(int i=0; i<n; i++ )
        for(int j=0; j<n; j++ )
            matriz[i][j] = null;
}
```



```
//      protected PositionList<Vertex<V>> vertices;
//      protected PositionList<Edge<E>> arcos;
//      protected Edge<E> [][] matriz;
//      protected int cantidadVertices;
```

```
public Vertex<V> insertVertex(V x) {
    Vertice<V> vv = new Vertice<V>(x, cantidadVertices++);
    vertices.addLast( vv );
    vv.setPosicionEnVertices( vertices.last() );
    return vv;
}
```



```
//      protected PositionList<Vertex<V>> vertices;
//      protected PositionList<Edge<E>> arcos;
//      protected Edge<E> [][] matriz;
//      protected int cantidadVertices;
```

$$T_{\text{insertEdge}}(n,m) = O(1)$$

```
public Edge<E> insertEdge(Vertex<V> v, Vertex<V> w, E x) {
```

```
    // Cargo arco en la matriz:
```

```
    Vertice<V> vv = (Vertice<V>) v;
```

```
    Vertice<V> ww = (Vertice<V>) w;
```

```
    int fila = vv.getIndice();
```

```
    int col = ww.getIndice();
```

```
    Arco<V,E> arco = new Arco<V,E>( x, vv, ww );
```

```
    // Grafo no dirigido => matriz simétrica
```

```
    matriz[fila][col] = matriz[col][fila] = arco;
```

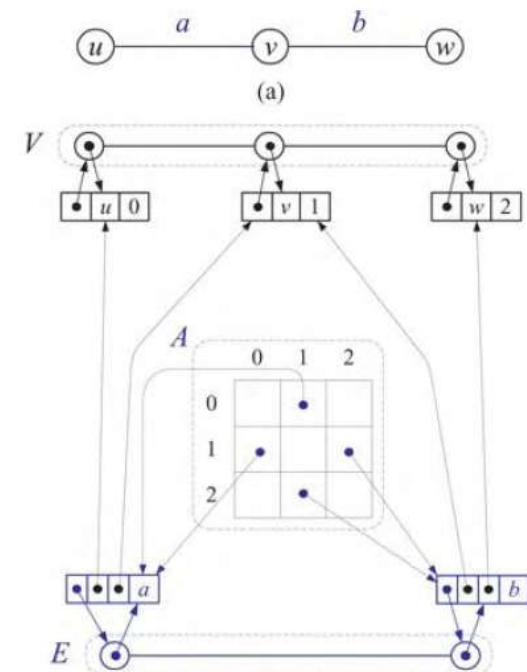
```
    // Cargo arco en la lista de arcos:
```

```
    arcos.addLast( arco );
```

```
    arco.setPosicionEnArcos( arcos.last() );
```

```
    return arco;
```

```
}
```



```
//      protected PositionList<Vertex<V>> vertices;
//      protected PositionList<Edge<E>> arcos;
//      protected Edge<E> [][] matriz;
//      protected int cantidadVertices;
```

```
public Iterable<Vertex<V>> vertices() {
    PositionList<Vertex<V>> lista = new DoubleLinkedList<Vertex<V>>();
    for( Vertex<V> v : vertices )
        lista.addLast(v);
    return lista;
}
```

$$T_{\text{vertices}}(n,m) = O(n)$$

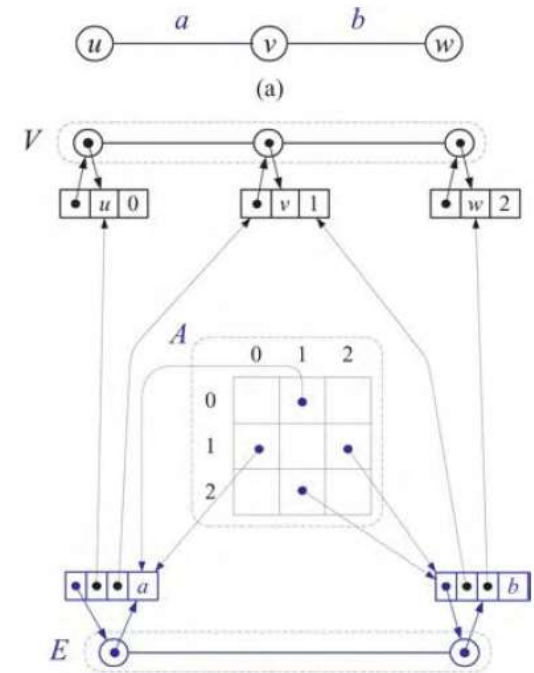
```
public Iterable<Edge<E>> edges() {
    ... similar a vértices...
}
```

```
//      protected PositionList<Vertex<V>> vertices;
//      protected PositionList<Edge<E>> arcos;
//      protected Edge<E> [][] matriz;
//      protected int cantidadVertices;
```

Hay que recorrer toda la fila i donde i =índice de v :

```
public Iterable<Edge<E>> incidentEdges(Vertex<V> v) {
    Vertice<V> vv = (Vertice<V>) v;
    int i = vv.getIndice();
    PositionList<Edge<E>> lista = new DoubleLinkedList<Edge<E>>();
    for( int j = 0; j < cantidadVertices; j++ )
        if( matriz[i][j] != null )
            lista.addLast( matriz[i][j] );
    return lista;
}
```

$T_{\text{incidentEdges}}(n,m) = O(n)$ porque el for-each hace n iteraciones y addLast de lista tiene $O(1)$ si la lista es doblemente enlazada con centinelas.



```
//      protected PositionList<Vertex<V>> vertices;
//      protected PositionList<Edge<E>> arcos;
//      protected Edge<E> [][] matriz;
//      protected int cantidadVertices;
```

Es similar al *opposite* de las otras representaciones:
Miro el arco y devuelvo el *otro* vértice diferente a v:

```
public Vertex<V> opposite(Vertex<V> v, Edge<E> e)
    throws GraphException {
    Arco<V,E> ee = (Arco<V,E>) e;
    if( ee.getV1() == v ) return ee.getV2();
    else if( ee.getV2() == v ) return ee.getV1();
    else throw new
        GraphException( "Vertice y arco no relacionados");
}
```

$$T_{\text{opposite}}(n,m) = O(1)$$

```
//      protected PositionList<Vertex<V>> vertices;
//      protected PositionList<Edge<E>> arcos;
//      protected Edge<E> [][] matriz;
//      protected int cantidadVertices;
```

Armo un arreglo con los dos vértices conectados por el arco en $O(1)$:

```
public Vertex<V> [] endVertices(Edge<E> e) {
    Vertex<V> [] a = (Vertex<V> []) new Vertice[2];
    Arco<V,E> ee = (Arco<V,E>) e;
    a[0] = ee.getV1();
    a[1] = ee.getV2();
    return a;
}
```

Los vértices v y w son adyacentes cuando hay un arco en la posición (i,j) , se hace en $O(1)$:

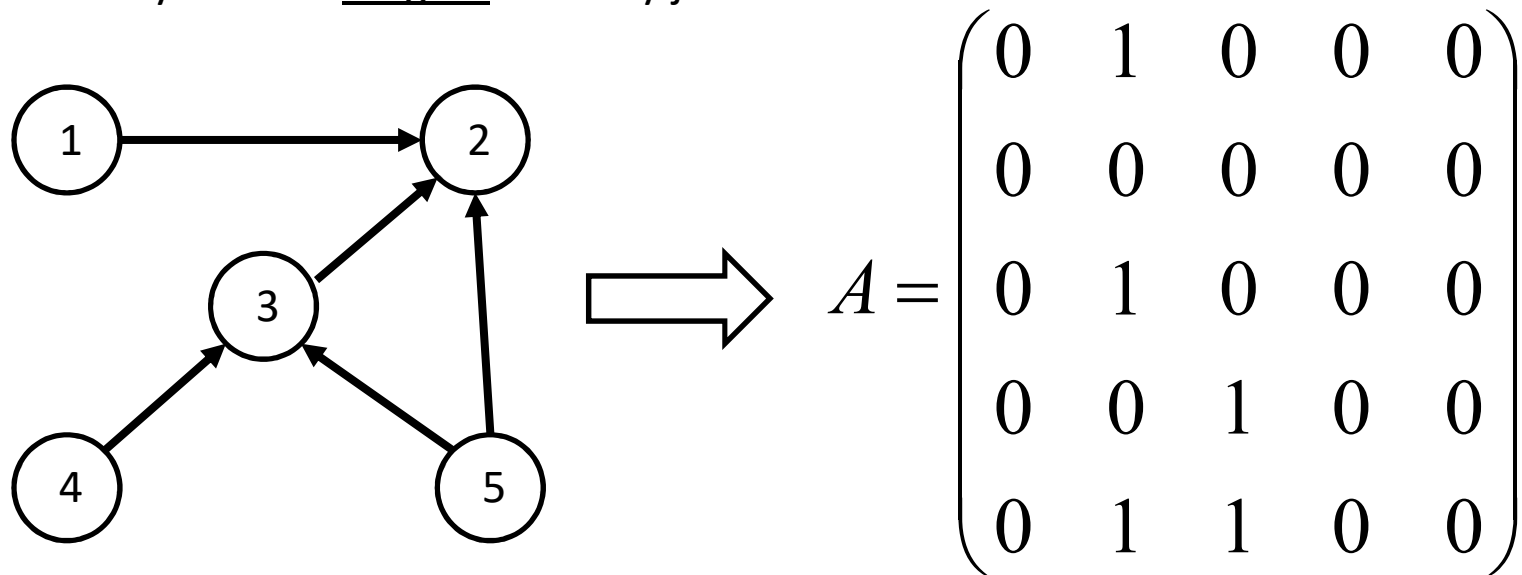
```
public boolean areAdjacent(Vertex<V> v, Vertex<V> w) {
    Vertice<V> vv = (Vertice<V>) v;
    Vertice<V> ww = (Vertice<V>) w;
    int i = vv.getIndice();
    int j = ww.getIndice();
    return matriz[i][j] != null;
}
```


Eliminar un arco requiere eliminarlo de la matriz de adyacencias y de la lista de arcos, todo en $O(1)$:

```
public E removeEdge(Edge<E> e) {  
    Arco<V,E> ee = (Arco<V,E>) e;  
    int fila = ee.getV1().getIndice();  
    int col = ee.getV2().getIndice();  
    matriz[fila][col] = null;  
    matriz[col][fila] = null;  
    arcos.remove( ee.getPosicionEnArcos() );  
    return e.element();  
}
```

Enfoques matemáticos para representar un grafo dirigido con una matriz de adyacencias

Un grafo G dirigido con vértices numéricos se puede representar matemáticamente por una matriz booleana $A=(a_{ij})$ con $i,j=1,\dots,n$ donde $a_{ij} = 1$ quiere decir que hay un arco dirigido entre los vértices i y j y $a_{ij} = 0$ quiere decir que no hay un arco dirigido entre i y j .



Sea $G=(V,E)$ donde:

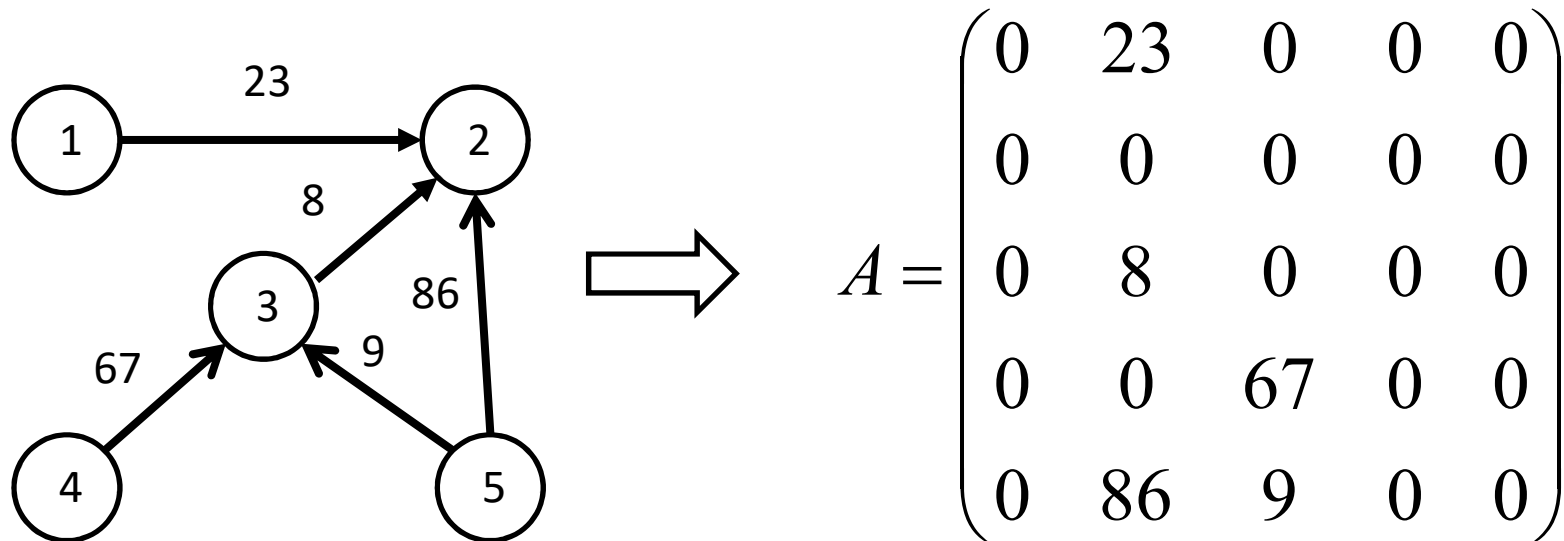
$V=\{1,2,3,4,5\}$

$E = \{ (1,2), (3,2), (4,3), (5,2), (5,3) \}$

Por ejemplo, como $(5,2)$ es un arco entonces $a_{52} = 1$. En el arco $(5,2)$, 5 es la *cola* del arco y 2 es la *punta*.

Enfoques matemáticos para representar un grafo dirigido pesado con una matriz de adyacencias

Un grafo G dirigido pesado con vértices numéricos se puede representar matemáticamente por una matriz real $A=(a_{ij})$ con $i,j=1,\dots,n$ donde a_{ij} es el peso del arco dirigido (i,j) .



Sea $G=(V,E)$ donde:

$V=\{1,2,3,4,5\}$

$E = \{(1,2), (3,2), (4,3), (5,2), (5,3)\}$

$w(1,2)=23, w(3,2)=8, w(5,2)=86,$

$w(4,3)=67, w(5,3)=9.$

Por ejemplo, como $(3,2)$ es un arco con peso 8 (notado como $w(3,2)=8$) entonces $a_{32} = 8$.

Modificaciones para representar grafos dirigidos con matriz de adyacencias

- Necesitamos una lista de vértices y una lista de arcos (dirigidos).
- La representación de los vértices es igual.
- La representación de los arcos asume que un vértice es la cola del arco y el otro es la punta.
- `insertEdge(v,w,x)` inserta un arco dirigido de `v` a `w`.
- `incidentEdges(v)` debe recorrer la columna de la matriz correspondiente al índice de `v`.
- `emergentEdges(v)` debe recorrer la fila de la matriz correspondiente al índice de `v`.
- `endVertices(e)` retorna un arreglo `a` donde `a[0] = cola(e)` y `a[1] = punta(e)`.

Modificaciones para representar multigrafos

- Repaso: Un multigrafo es un grafo donde puede haber más de un arco conectando un par de vértices.
- Con lista de arcos: Lo soporta naturalmente.
- Con lista de adyacencias: Lo soporta naturalmente.
- Con matriz de adyacencias: La componente (i,j) de la matriz debe ser una lista de arcos.

Bibliografía

- Capítulo 13 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.